

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«Омский государственный технический университет»

Факультет информационных технологий и компьютерных систем
Кафедра «Прикладная математика и фундаментальная информатика»

ДОМАШНЕЕ ЗАДАНИЕ

по дисциплине Практикум по программированию
на тему Разработка Data Science проекта

Студента Рау Алексея Евгеньевича
фамилия, имя, отчество полностью

Курс 3 Группа ФИТ-231

Направление 02.03.02 «Фундаментальная информатика и
информационные технологии»
код, наименование

Руководитель ассистент
должность, ученая степень, звание
Бугаенко Иван Евгеньевич
фамилия, инициалы

Выполнил _____
дата, подпись студента

_____ баллы _____ дата, подпись руководителя



Оглавление

- Описание задачи
- Описание исходных данных
- - 1. Предварительная обработка данных
 - 1.1. Чтение и загрузка данных
 - 1.2. Первичный анализ данных
 - 1.2.1. Приведение пропусков к общему виду
 - 1.2.2. Удаление неинформативных признаков
 - 1.3. Разделение выборки на обучающую и тестовую выборки
 - 1.4. Кодирование категориальных признаков
 - 1.5. Обработка вещественных признаков (заполнение пропусков)
 - 1.5.1. Анализ пропусков в обучающей выборке
 - 1.5.2. Статистический подход (KL-дивергенция)
 - 1.5.3. Машинный подход (KNNImputer)
 - 1.6. Детекция выбросов и аномалий в данных
 - 1.6.1. Поиск и удаление выбросов
 - 1.6.1.1. Анализ распределений и визуализация "Ящик с усами"
 - 1.6.1.2. Реализация методов IQR и Robust Z-score
 - 1.6.1.3. Поиск выбросов с использованием обоих методов
 - 1.6.1.4. Удаление объектов с выбросами
 - 1.6.2. Поиск и удаление аномалий
 - 1.6.2.1. Подготовка данных и снижение размерности
 - 1.6.2.2. Применение Isolation Forest
 - 1.6.2.3. Применение One-Class SVM
 - 1.6.2.4. Применение DBSCAN
 - 1.6.2.5. Определение общих аномалий
 - 1.6.2.6. Удаление аномалий и итоговая статистика
 - 1.7. Подведение итогов раздела 1
 - 2. Генерация новых признаков

- 2.1. Корреляционный анализ входных признаков (построение тепловых карт корреляции)
 - 2.2. Исправление проблемы мультиколлинеарности в данных
 - 2.3. Скалирование данных
 - 2.4. Подведение итогов раздела 2
- 3. Выбор моделей ML и метрик
- 4. Обучение моделей ML и подбор гиперпараметров
- 5. Оценка моделей и выбор лучшей
- 6. Результат работы

Описание задачи

Задача: Прогнозирование коэффициента Гармония Бессмертия — ключевого показателя состояния магического портала "Врата Вечности".

Тип задачи: Задача **регрессии**, так как целевая переменная Гармония Бессмертия является непрерывной числовой величиной (коэффициент в диапазоне от 0 до 1).

Цель: Построить модель машинного обучения, которая по параметрам работы портала сможет предсказать значение его Гармонии.

Описание исходных данных

№	Признак	Описание	Ожидаемый тип данных
Целевые признаки			
1	Гармония Бессмертия	Коэффициент, указывающий на состояние магического ядра портала и его стабильность	Вещественный
Входные признаки			
2	Вектор Мощи	Позиция магического механизма, управляющего интенсивностью перехода через портал	Вещественный
3	Скорость перехода через портал	Скорость, с которой портал перемещает объект через пространство	Вещественный

№	Признак	Описание	Ожидаемый тип данных
4	Приток Силы Потока	Поток магической силы, текущий через ядро портала, обеспечивая стабильную работу	Вещественный
5	Ритм магического ядра	Число оборотов магического ядра портала в минуту	Вещественный
6	Поток Энергий	Число оборотов генератора эфира, преобразующего внешнюю энергию в магическую	Вещественный
7	Сила Левого Потока	Мощность магического потока, текущего через левую сторону портала	Вещественный
8	Сила Правого Потока	Мощность магического потока, текущего через правую сторону портала	Вещественный
9	Пламя Стихий	Температура магической энергии, исходящей из высокого магического источника портала (°C)	Вещественный
10	Температура вдоха Истока	Температура воздуха, входящего в магический ускоритель портала (°C)	Вещественный
11	Температура выдоха Истока	Температура воздуха, исходящего из магического ускорителя портала (°C)	Вещественный
12	Приток давления Выдоха Истока	Давление магического потока на выходе из магического источника высокого давления	Вещественный
13	Давление вдоха Истока	Давление воздуха, входящего в магический ускоритель	Вещественный
14	Давление выдоха Истока	Давление воздуха, исходящего из магического ускорителя	Вещественный
15	Древний Ветер	Давление древней магической энергии, покидающей портал в виде выхлопного потока	Вещественный
16	Печать Чародея	Параметр управления магическим выбросом в сердце портала (%)	Вещественный
17	Эмульсия Истока	Количество магического топлива, подпитывающего портал	Вещественный
18	Дыхание Истока	Коэффициент, отображающий степень угасания магического ускорителя	Вещественный
19	Тип Вектора	Метка типа позиции магического механизма	Категориальный

№	Признак	Описание	Ожидаемый тип данных
	Мощи		
20	Номер пометки	Уникальный номер записи о работе портала	Целочисленный

1. Предварительная обработка данных

```
In [121... import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib
import os
import optuna
import sklearn
import warnings
warnings.filterwarnings('ignore')
optuna.logging.set_verbosity(optuna.logging.WARNING)
from scipy import stats
from scipy.stats import entropy
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.impute import KNNImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score,
from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler
from sklearn.ensemble import IsolationForest
from sklearn.svm import OneClassSVM, SVR
from sklearn.cluster import DBSCAN
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from umap import UMAP
warnings.filterwarnings('ignore')
```

```
In [122... seed = 42
np.random.seed(seed)
```

```
In [123... print("Версии:")
print(f"Python: {sys.version.split()[0]}")
print(f"Pandas: {pd.__version__}")
print(f"NumPy: {np.__version__}")
print(f"Scikit-learn: {sklearn.__version__}")
print(f"Optuna: {optuna.__version__}")
print(f"SEED зафиксирован: {seed}")
```

Версии:
Python: 3.12.6
Pandas: 2.3.3
NumPy: 2.3.5
Scikit-learn: 1.7.2
Optuna: 4.6.0
SEED зафиксирован: 42

1.1. Чтение и загрузка данных

```
In [124...] df = pd.read_csv('portal_data.csv', encoding='windows-1251', sep='|')
```

Вывод первых строк для проверки:

```
In [125...] df.head()
```

Out[125...]

	Вектор Мощи	Скорость перехода через портал	Приток Силы Потока	Ритм магического ядра	Поток Энергий	Сила Левого Потока	Сила Правого Потока
0	1.138	1.54332	289.964	141.318158	6677.380	7.584	7.584
1	2.088	-	6960.180	144.111767	6828.469	28.204	28.204
2	3.144	4.6299600000000005	8379.229	145.220853	7111.811	60.358	60.358
3	4.161	6.17328	14724.395	162.050156	7792.630	113.774	113.774
4	5.140	7.7166	21636.432	201.513586	8494.777	175.306	175.306

1.2. Первичный анализ данных

Общая информация о датасете:

```
In [126...] df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11934 entries, 0 to 11933
Data columns (total 20 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Вектор Мощи                           11934 non-null  float64
1   Скорость перехода через портал        11934 non-null  object
2   Приток Силы Потока                   11934 non-null  float64
3   Ритм магического ядра                 11934 non-null  float64
4   Поток Энергий                         11934 non-null  float64
5   Сила Левого Потока                   11934 non-null  float64
6   Сила Правого Потока                   11934 non-null  float64
7   Пламя Стихий                         11934 non-null  float64
8   Температура вдоха Истока              11934 non-null  int64
9   Температура выдоха Истока             11934 non-null  object
10  Приток давления Выдоха Истока         11934 non-null  float64
11  Давление вдоха Истока                  11934 non-null  object
12  Давление выдоха Истока                 11934 non-null  float64
13  Древний Ветер                         11934 non-null  object
14  Печать Чародея                        11934 non-null  float64
15  Эмульсия Истока                       11934 non-null  float64
16  Дыхание Истока                        11934 non-null  float64
17  Гармония Бессмертия                   11934 non-null  float64
18  Тип Вектора Мощи                      11934 non-null  object
19  Номер пометки                         11934 non-null  int64
dtypes: float64(13), int64(2), object(5)
memory usage: 1.8+ MB
```

In [127... `df.describe()`

Out[127...

	Вектор Мощи	Приток Силы Потока	Ритм магического ядра	Поток Энергий	Сила Левого Потока	11
count	11934.000000	11934.000000	11934.000000	11934.000000	11934.000000	11
mean	5.166667	27247.498685	223.711688	8200.947312	227.335768	
std	2.626388	22148.613155	81.061874	1091.315507	200.495889	
min	1.138000	253.547000	136.939406	6589.002000	5.304000	
25%	3.144000	8375.883750	145.220958	7058.324000	60.317000	
50%	5.140000	21630.659000	201.514947	8482.081500	175.268000	
75%	7.148000	39001.426750	280.447777	9132.606000	332.364750	
max	9.300000	72784.872000	372.879926	9797.103000	645.249000	

Рассмотрим датасет на содержание уникальных значений:

In [128... `df.nunique()`

```
Out[128... Вектор Мощи 9
Скорость перехода через портал 10
Приток Силы Потока 11430
Ритм магического ядра 3888
Поток Энергий 11834
Сила Левого Потока 4286
Сила Правого Потока 4286
Пламя Стихий 11772
Температура вдоха Истока 1
Температура выдоха Истока 2369
Приток давления Выдоха Истока 524
Давление вдоха Истока 2
Давление выдоха Истока 4209
Древний Ветер 20
Печать Чародея 8496
Эмульсия Истока 696
Дыхание Истока 51
Гармония Бессмертия 26
Тип Вектора Мощи 4
Номер пометки 11934
dtype: int64
```

Анализ категориального признака:

```
In [129... categorical_column = 'Тип Вектора Мощи'
print(f"Категориальный признак '{categorical_column}':")
print(f"Количество уникальных значений: {df[categorical_column].nunique()}")
print(f"Уникальные значения: {df[categorical_column].unique()}")
print(f"Распределение значений:")
print(df[categorical_column].value_counts())
```

Категориальный признак 'Тип Вектора Мощи':

Количество уникальных значений: 4

Уникальные значения: ['Слабый' 'Ниже среднего' 'Выше среднего' 'Сильный']

Распределение значений:

Тип Вектора Мощи

Сильный 3978

Слабый 2652

Ниже среднего 2652

Выше среднего 2652

Name: count, dtype: int64

Сравнение индентичных на первый взгляд признаков:

```
In [130... mismatch_count = (df['Сила Левого Потока'] != df['Сила Правого Потока']).sum()
print(f"Количество несовпадений: {mismatch_count}")
```

Количество несовпадений: 0

Проблемы, выявленные при анализе:

- Несколько столбцов имеют тип object вместо числового типа из-за наличия строковых значений ("-", "Не определено")

- Признак `Температура вдоха Истока` имеет только 1 уникальное значение
- Признак `Номер пометки` имеет 11934 уникальных значения (идентификатор)
- `Сила Левого Потока` и `Сила Правого Потока` имеют абсолютно одинаковые значения
- Аномальное значение в столбце `Скорость перехода через портал` (4.6299600000000005)

1.2.1. Приведение пропусков к общему виду

Поскольку несколько столбцов имеют тип `object` из-за наличия строковых значений "-" и "Не определено". Преобразуем их в числовой формат:

```
In [131... object_columns = [
    'Скорость перехода через портал',
    'Температура выдоха Истока',
    'Давление вдоха Истока',
    'Древний Ветер'
]

for col in object_columns:
    df[col] = pd.to_numeric(df[col].replace(['-', 'Не определено'], np.nan), errors='coerce')

print("Типы данных после преобразования:")
print(df[object_columns].dtypes)
print(f"\nКоличество пропусков после преобразования:")
print(df[object_columns].isnull().sum())
```

Типы данных после преобразования:

Скорость перехода через портал	float64
Температура выдоха Истока	float64
Давление вдоха Истока	float64
Древний Ветер	float64
dtype:	object

Количество пропусков после преобразования:

Скорость перехода через портал	1203
Температура выдоха Истока	9547
Давление вдоха Истока	9495
Древний Ветер	9547
dtype:	int64

Теперь можно рассмотреть количество пропусков в каждом признаке

```
In [132... (df.isna().sum()/df.shape[0]*100).sort_values(ascending=False)
```

```
Out[132... Древний Ветер 79.998324
Температура выдоха Истока 79.998324
Давление вдоха Истока 79.562594
Скорость перехода через портал 10.080442
Ритм магического ядра 0.000000
Приток Силы Потока 0.000000
Вектор Мощи 0.000000
Поток Энергий 0.000000
Пламя Стихий 0.000000
Сила Правого Потока 0.000000
Температура вдоха Истока 0.000000
Сила Левого Потока 0.000000
Приток давления Выдоха Истока 0.000000
Давление выдоха Истока 0.000000
Печать Чародея 0.000000
Эмульсия Истока 0.000000
Дыхание Истока 0.000000
Гармония Бессмертия 0.000000
Тип Вектора Мощи 0.000000
Номер пометки 0.000000
dtype: float64
```

1.2.2. Удаление неинформативных признаков

Удаляем признаки, которые не несут информационной ценности для модели

1. Номер пометки - уникальный идентификатор
2. Температура вдоха Истока - имеет только 1 уникальное значение
3. Сила Правого Потока - полностью дублирует Сила Левого Потока
4. Древний Ветер , Температура выдоха Истока , Давление вдоха Истока имеют около 80% пропущенных значений

```
In [133... columns_to_drop = ['Номер пометки', 'Температура вдоха Истока', 'Сила Правого  
df = df.drop(columns=columns_to_drop)
```

Данные после обработки:

```
In [134... df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11934 entries, 0 to 11933
Data columns (total 14 columns):
#   Column                                          Non-Null Count  Dtype
---  -
0   Вектор Мощи                                   11934 non-null  float64
1   Скорость перехода через портал              10731 non-null  float64
2   Приток Силы Потока                           11934 non-null  float64
3   Ритм магического ядра                       11934 non-null  float64
4   Поток Энергий                               11934 non-null  float64
5   Сила Левого Потока                           11934 non-null  float64
6   Пламя Стихий                                11934 non-null  float64
7   Приток давления Выдоха Истока              11934 non-null  float64
8   Давление выдоха Истока                      11934 non-null  float64
9   Печать Чародея                              11934 non-null  float64
10  Эмульсия Истока                             11934 non-null  float64
11  Дыхание Истока                             11934 non-null  float64
12  Гармония Бессмертия                         11934 non-null  float64
13  Тип Вектора Мощи                            11934 non-null  object
dtypes: float64(13), object(1)
memory usage: 1.3+ MB

```

In [135... df.head(10)

Out[135...

	Вектор Мощи	Скорость перехода через портал	Приток Силы Потока	Ритм магического ядра	Поток Энергий	Сила Левого Потока	Пламя Стихий	П дав В V
0	1.138	1.54332	289.964	141.318158	6677.380	7.584	464.006	
1	2.088	NaN	6960.180	144.111767	6828.469	28.204	635.401	
2	3.144	4.62996	8379.229	145.220853	7111.811	60.358	606.002	
3	4.161	6.17328	14724.395	162.050156	7792.630	113.774	661.471	
4	5.140	7.71660	21636.432	201.513586	8494.777	175.306	731.494	
5	6.175	9.25992	29792.731	241.630782	8828.360	246.278	800.434	
6	7.148	10.80324	38982.180	280.448510	9132.429	332.077	854.747	
7	8.206	12.34656	50996.808	323.328632	9318.562	437.989	952.122	
8	9.300	13.88988	72763.329	372.843693	9778.528	644.905	1115.797	
9	1.138	1.54332	379.880	141.934538	6683.916	7.915	464.017	

1.3. Разделение выборки на обучающую и тестовую выборки

Выделение целевого признака и предикторов

```
In [136... target_column = 'Гармония Бессмертия'
X = df.drop(columns=[target_column])
y = df[target_column]
```

Объект признак:

```
In [137... print("матрица объект-признак:")
X
```

матрица объект-признак:

```
Out[137...
```

	Вектор Мощи	Скорость перехода через портал	Приток Силы Потока	Ритм магического ядра	Поток Энергий	Сила Левого Потока	Пламя Стихий
0	1.138	1.54332	289.964	141.318158	6677.380	7.584	464.006
1	2.088	NaN	6960.180	144.111767	6828.469	28.204	635.401
2	3.144	4.62996	8379.229	145.220853	7111.811	60.358	606.002
3	4.161	6.17328	14724.395	162.050156	7792.630	113.774	661.471
4	5.140	7.71660	21636.432	201.513586	8494.777	175.306	731.494
...	...	...	...	...	...	...	...
11929	5.140	7.71660	21624.934	201.516623	8470.013	175.239	681.658
11930	6.175	9.25992	29763.213	241.561772	8800.352	245.954	747.405
11931	7.148	10.80324	39003.867	280.444950	9120.889	332.389	796.457
11932	8.206	12.34656	50992.579	323.315332	9300.274	438.024	892.945
11933	9.300	13.88988	72775.130	372.844216	9742.950	644.880	1038.411

11934 rows × 13 columns

Целевые значения:

```
In [138... print("список целевых значений:")
y
```

список целевых значений:

```
Out[138... 0      0.975
          1      0.975
          2      0.975
          3      0.975
          4      0.975
          ...
        11929    1.000
        11930    1.000
        11931    1.000
        11932    1.000
        11933    1.000
Name: Гармония Бессмертия, Length: 11934, dtype: float64
```

Разбиение на обучающую и тестовую выборки

```
In [139... X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

80/20 - стандартное соотношение, обеспечивающее достаточное количество данных для обучения

```
In [140... print(f"\nРазмеры выборок:")
print(f"Обучающая: {X_train.shape}, {y_train.shape}")
print(f"Тестовая: {X_test.shape}, {y_test.shape}")
```

Размеры выборок:
Обучающая: (9547, 13), (9547,)
Тестовая: (2387, 13), (2387,)

1.4. Кодирование категориальных признаков

Из ранее полученных данных видно, что единственный категориальный признак — Тип Вектора Мощи .

Его уникальные значения:

- Слабый
- Ниже среднего
- Выше среднего
- Сильный

Категории имеют отношение порядка (от слабого к сильному), поэтому подходит **Ordinal Encoding**.

Создадим словарь соответствия

```
In [141... order_mapping = {
    'Слабый': 0,
    'Ниже среднего': 1,
```

```

    'Выше среднего': 2,
    'Сильный': 3
}

```

Применение Ordinal Encoding к обучающей выборке

```

In [142... categorical_column = 'Тип Вектора Мощи'

print("Категориальный признак до кодирования:")
print(f"Уникальные значения: {X_train[categorical_column].unique()}")
print(f"Распределение значений:")
print(X_train[categorical_column].value_counts())

X_train['Тип Вектора Мощи'] = X_train['Тип Вектора Мощи'].map(order_mapping)

print("\nКатегориальный признак после кодирования:")
print(f"Уникальные значения: {X_train[categorical_column].unique()}")
print(f"Распределение значений:")
print(X_train[categorical_column].value_counts().sort_index())

print(f"Тип данных: {X_train[categorical_column].dtype}")

```

Категориальный признак до кодирования:
 Уникальные значения: ['Выше среднего' 'Слабый' 'Сильный' 'Ниже среднего']
 Распределение значений:
 Тип Вектора Мощи

Сильный	3182
Слабый	2147
Ниже среднего	2121
Выше среднего	2097

Name: count, dtype: int64

Категориальный признак после кодирования:
 Уникальные значения: [2 0 3 1]
 Распределение значений:
 Тип Вектора Мощи

0	2147
1	2121
2	2097
3	3182

Name: count, dtype: int64
 Тип данных: int64

Применение Ordinal Encoding к тестовой выборке

```

In [143... print("Категориальный признак до кодирования:")
print(f"Уникальные значения: {X_test[categorical_column].unique()}")
print(f"Распределение значений:")
print(X_test[categorical_column].value_counts())

X_test['Тип Вектора Мощи'] = X_test['Тип Вектора Мощи'].map(order_mapping)

print("\nКатегориальный признак после кодирования:")

```

```
print(f"Уникальные значения: {X_test[categorical_column].unique()}")
print(f"Распределение значений:")
print(X_test[categorical_column].value_counts().sort_index())

print(f"Тип данных: {X_test[categorical_column].dtype}")
```

Категориальный признак до кодирования:

Уникальные значения: ['Ниже среднего' 'Сильный' 'Выше среднего' 'Слабый']

Распределение значений:

Тип Вектора Мощи

Сильный 796

Выше среднего 555

Ниже среднего 531

Слабый 505

Name: count, dtype: int64

Категориальный признак после кодирования:

Уникальные значения: [1 3 2 0]

Распределение значений:

Тип Вектора Мощи

0 505

1 531

2 555

3 796

Name: count, dtype: int64

Тип данных: int64

1.5. Обработка вещественных признаков (заполнение пропусков)

1.5.1. Анализ пропусков в обучающей выборке

```
In [144... missing_cols = X_train.columns[X_train.isna().any()].tolist()
print("Признаки с пропусками в обучающей выборке:")
for col in missing_cols:
    missing_percent = (X_train[col].isna().sum() / len(X_train)) * 100
    print(f" - {col}: {missing_percent:.2f}% пропусков")
```

Признаки с пропусками в обучающей выборке:

- Скорость перехода через портал: 10.07% пропусков

1.5.2. Статистический подход (KL-дивергенция)

Идея:

- Для каждого признака с пропусками определить пул значений (среднее, медиана, мода, константы).
- Построить гистограммы до и после заполнения.
- Рассчитать KL-дивергенцию между распределениями.

- Выбрать значение с минимальной KL-дивергенцией.

```
In [145... def kl_divergence(p, q):
    p = np.array(p) / np.sum(p)
    q = np.array(q) / np.sum(q)
    return entropy(p, q)
```

```
In [146... def evaluate_imputation(series, fill_value):
    original = series.dropna()
    filled = series.fillna(fill_value)

    plt.figure(figsize=(10, 6))

    plt.subplot(1, 2, 1)
    plt.hist(original, bins=9, edgecolor='black', alpha=1)
    plt.title('До заполнения')
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    plt.hist(filled, bins=9, edgecolor='black', alpha=1)
    plt.title('После заполнения')
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

    hist_original, _ = np.histogram(original, bins=20, density=True)
    hist_filled, _ = np.histogram(filled, bins=20, density=True)

    kl = kl_divergence(hist_original, hist_filled)
    return kl, hist_original, hist_filled
```

```
In [147... best_values = {}
for col in missing_cols:
    print(f"\n--- Обработка признака: {col} ---")

    candidate_values = [
        X_train[col].mean(),
        X_train[col].median(),
        X_train[col].mode()[0]
    ]

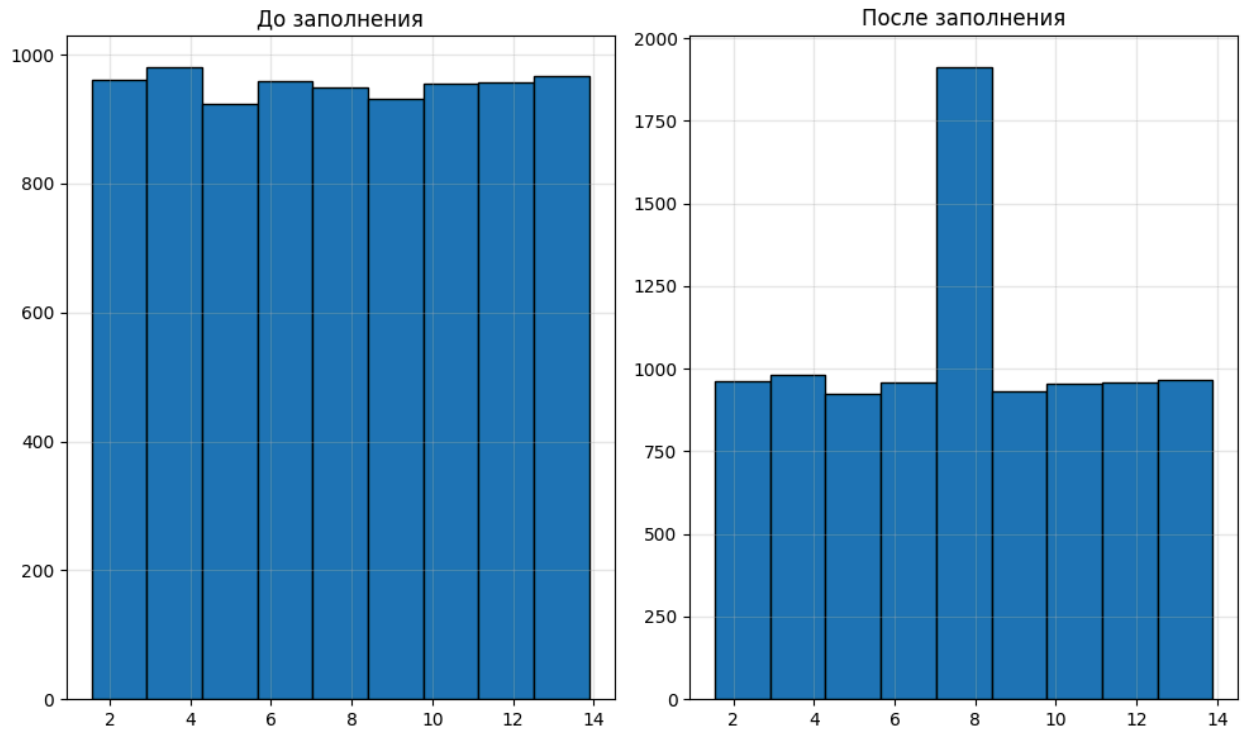
    best_kl = float('inf')
    best_val = None

    for val in candidate_values:
        kl, _, _ = evaluate_imputation(X_train[col], val)
        print(f"    Заполнение значением {val:.4f}: KL = {kl:.4f}")
        if kl < best_kl:
            best_kl = kl
            best_val = val
```

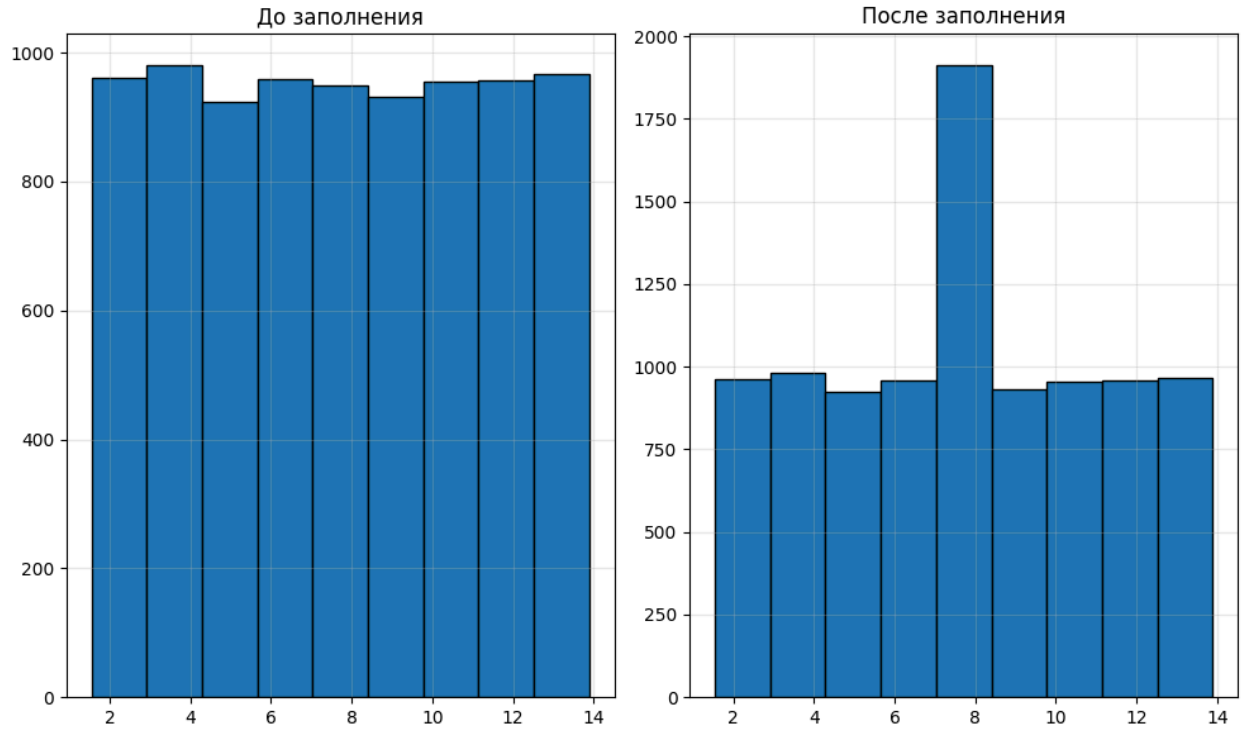


```
best_values[col] = best_val
print(f" Выбрано значение: {best_val:.4f} (KL = {best_kl:.4f})")
```

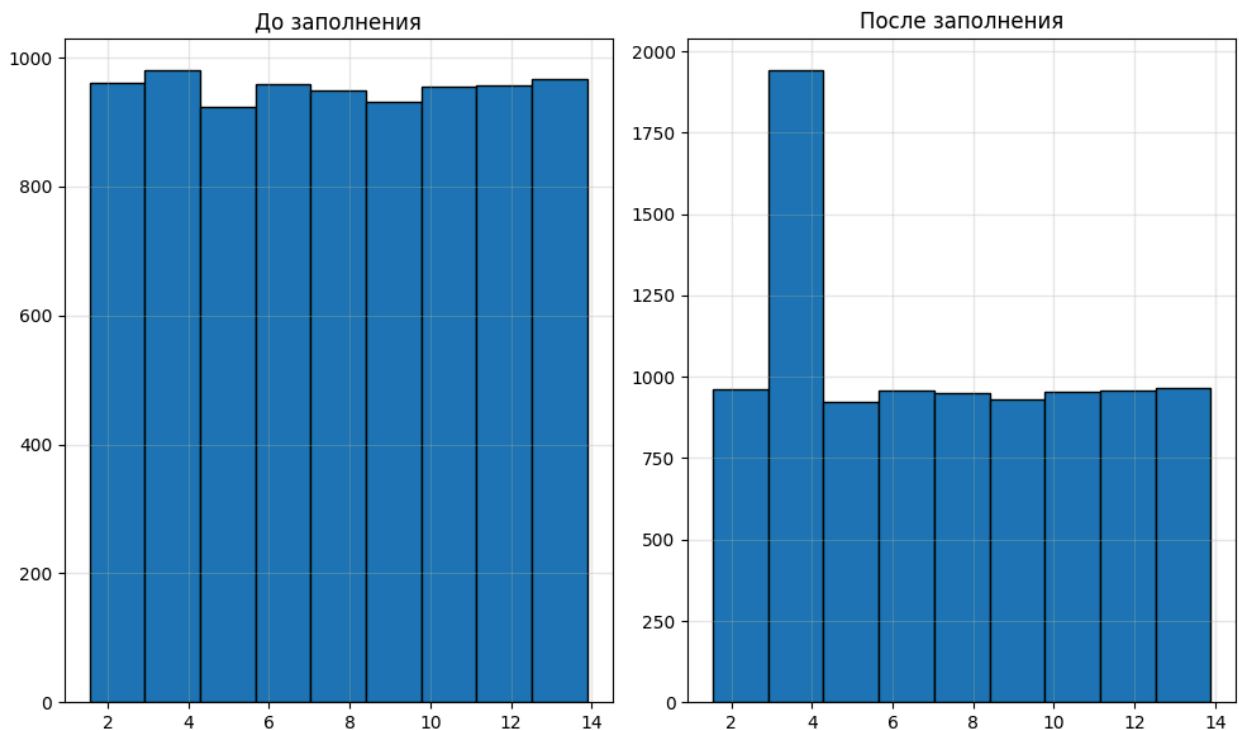
--- Обработка признака: Скорость перехода через портал ---



Заполнение значением 7.7152: KL = 0.1061



Заполнение значением 7.7166: KL = 0.0288



Заполнение значением 3.0866: KL = 0.0281

Выбрано значение: 3.0866 (KL = 0.0281)

1.5.3. Машинный подход (KNNImputer)

```
In [148... imputer = KNNImputer(n_neighbors=5)
X_train_knn = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.col

for col in missing_cols:

    print(f"\n--- Обработка признака: {col} ---")

    plt.figure(figsize=(10, 6))

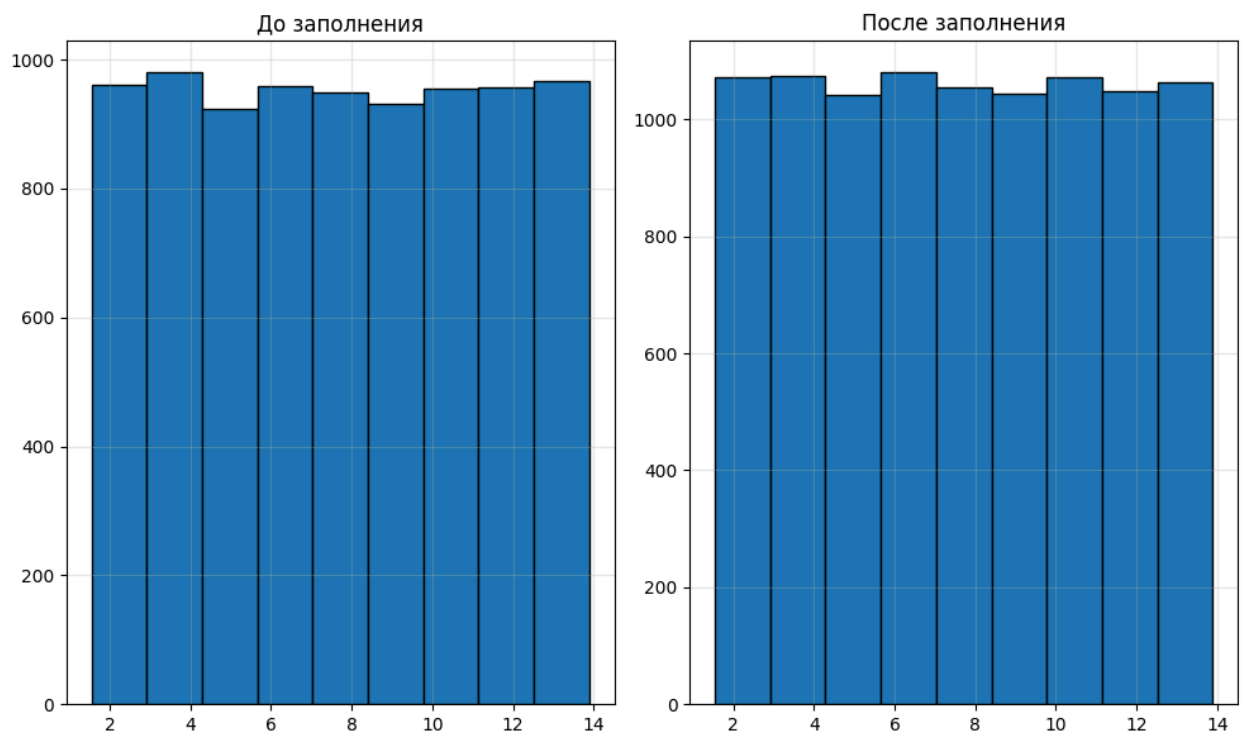
    plt.subplot(1, 2, 1)
    plt.hist(X_train[col].dropna(), bins=9, edgecolor='black', alpha=1)
    plt.title('До заполнения')
    plt.grid(True, alpha=0.3)

    plt.subplot(1, 2, 2)
    plt.hist(X_train_knn[col], bins=9, edgecolor='black', alpha=1)
    plt.title('После заполнения')
    plt.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.show()

print("Пропуски в обучающей выборке после KNNImputer:")
print(X_train_knn.isna().sum())
```

--- Обработка признака: Скорость перехода через портал ---



Пропуски в обучающей выборке после KNNImputer:

Вектор Мощи	0
Скорость перехода через портал	0
Приток Силы Потока	0
Ритм магического ядра	0
Поток Энергий	0
Сила Левого Потока	0
Пламя Стихий	0
Приток давления Выдоха Истока	0
Давление выдоха Истока	0
Печать Чародея	0
Эмульсия Истока	0
Дыхание Истока	0
Тип Вектора Мощи	0

dtype: int64

Применим результат к тестовой выборке

```
In [149... X_test_knn = pd.DataFrame(imputer.transform(X_test), columns=X_test.columns, i
```

1.6. Детекция выбросов и аномалий в данных

1.6.1. Поиск и удаление выбросов

1.6.1.1. Анализ распределений и визуализация "Ящик с усами"

Визуализация распределений для выявления выбросов

```
In [150... def draw_sns(df, visualization_function, figsize=(20, 20), **kwargs):
    n_col = 3
    n = len(df.columns)
    n_row = int(np.ceil(n / n_col))

    fig, axes = plt.subplots(n_row, n_col, figsize=figsize)
    axes = axes.flatten()

    for i, col in enumerate(df.columns):
        if visualization_function == sns.boxplot:
            visualization_function(y=df[col], ax=axes[i], **kwargs)
        else:
            visualization_function(x=df[col], ax=axes[i], **kwargs)

        axes[i].set_title(col, fontsize=12)
        axes[i].tick_params(axis='x', rotation=45, labelsz=10)
        axes[i].tick_params(axis='y', labelsz=10)
        axes[i].grid(True, alpha=0.3, linestyle='--')

    for j in range(i+1, len(axes)):
        axes[j].set_visible(False)

    plt.tight_layout()
    plt.show()

print("\nГистограммы распределений признаков:")
draw_sns(
    X_train_knn.select_dtypes(include=[np.number]),
    sns.histplot,
    figsize=(20, 20),
    alpha=1,
    edgecolor='black',
    linewidth=1.5,
    color='#5799c6'
)

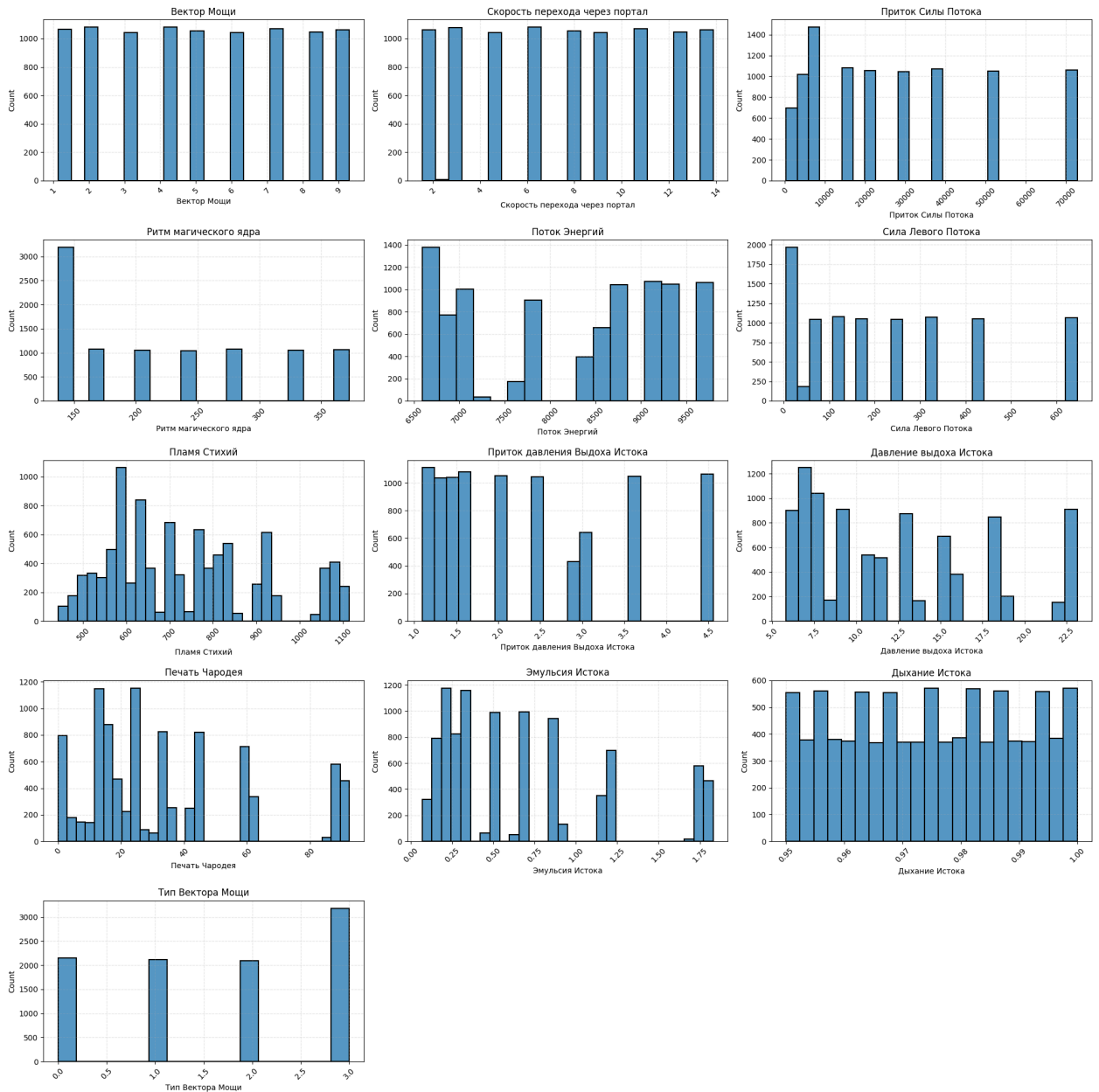
print("\nДиаграммы 'Ящик с усами' для выявления выбросов:")
draw_sns(
    X_train_knn.select_dtypes(include=[np.number]),
    sns.boxplot,
    figsize=(20, 25),
```

```

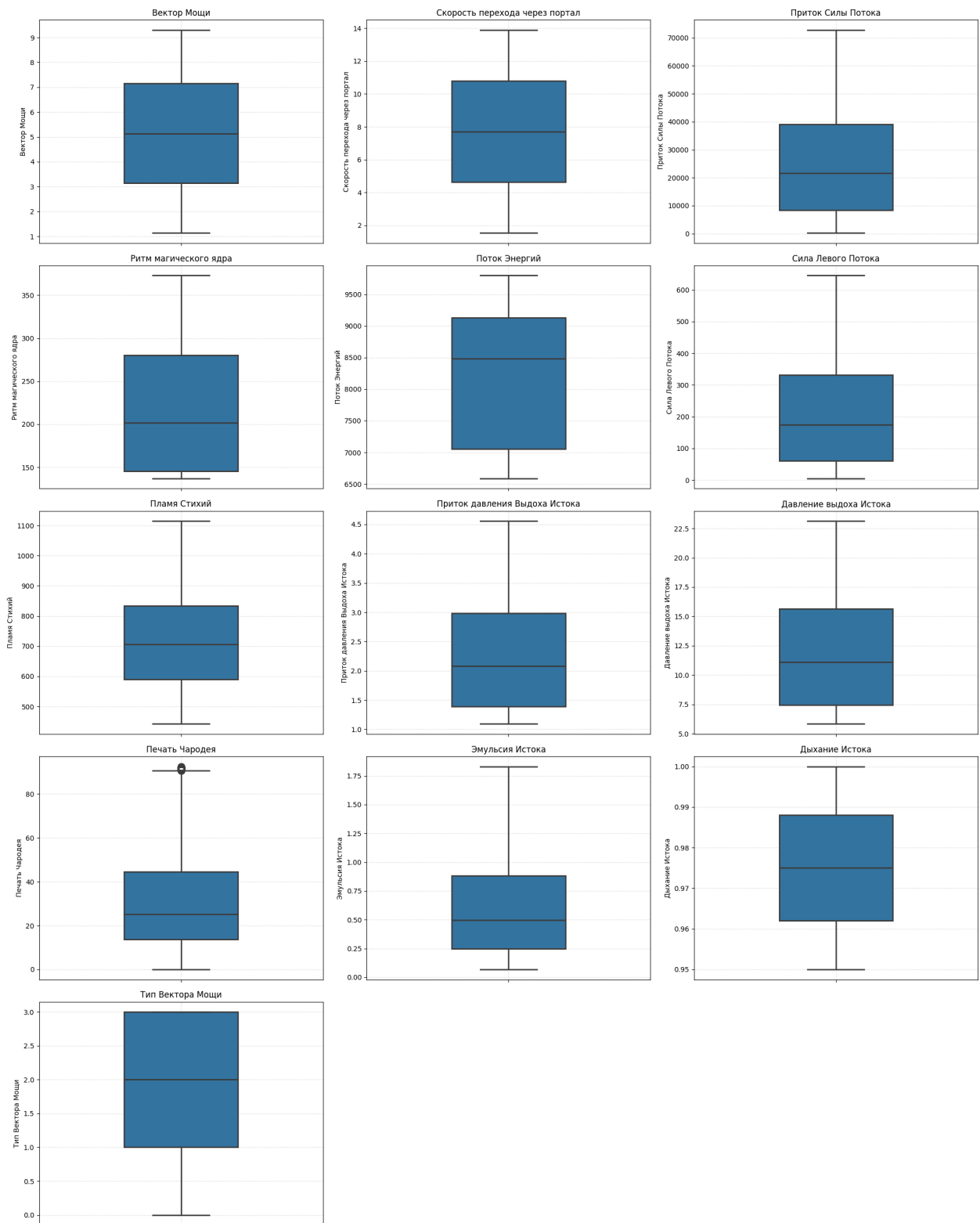
color='#3274a1',
saturation=1,
width=0.4,
fliersize=10,
linewidth=2
)

```

Гистограммы распределений признаков:



Диаграммы 'Ящик с усами' для выявления выбросов:



1.6.1.2. Реализация методов IQR и Robust Z-score

Функция для вычисления границ IQR

```
In [151... def calc_bounds_iqr(x):
```

```

q1, q3 = np.percentile(x, [25, 75])
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
return lower_bound, upper_bound

```

Функция для проверки выбросов методом IQR

```

In [152... def check_feature_outliers_iqr(x):
    lower_bound, upper_bound = calc_bounds_iqr(x)
    return ((x < lower_bound) | (x > upper_bound)).values

```

Функция для проверки выбросов методом Robust Z-score

```

In [153... def check_feature_outliers_robust_zscore(x, threshold=3):
    median = np.median(x)
    mad = np.median(np.abs(x - median))

    if mad == 0:
        mad = np.std(x)
        if mad == 0:
            return np.zeros(len(x), dtype=bool)

    robust_z_scores = 0.6745 * (x - median) / mad
    return (np.abs(robust_z_scores) > threshold).values

```

Функция для поиска объектов с выбросами в датасете

```

In [154... def check_dataset_outliers(df, outliers_method, **kwargs):
    outliers_flags = []
    for column in df.select_dtypes(include=[np.number]).columns:
        outliers_flags.append(outliers_method(df[column], **kwargs))

    return np.any(np.array(outliers_flags), axis=0)

```

1.6.1.3. Поиск выбросов с использованием обоих методов

```

In [155... outliers_iqr = check_dataset_outliers(X_train_knn, check_feature_outliers_iqr)
outliers_robust = check_dataset_outliers(X_train_knn, check_feature_outliers_r

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

iqr_counts = [np.sum(outliers_iqr), len(outliers_iqr) - np.sum(outliers_iqr)]
axes[0].pie(iqr_counts, labels=['Выбросы', 'Нормальные'], autopct='%1.1f%%')
axes[0].set_title('Метод IQR')

robust_counts = [np.sum(outliers_robust), len(outliers_robust) - np.sum(outliers_robust)]
axes[1].pie(robust_counts, labels=['Выбросы', 'Нормальные'], autopct='%1.1f%%')
axes[1].set_title('Метод Robust Z-score')

intersection_outliers = outliers_iqr & outliers_robust

```

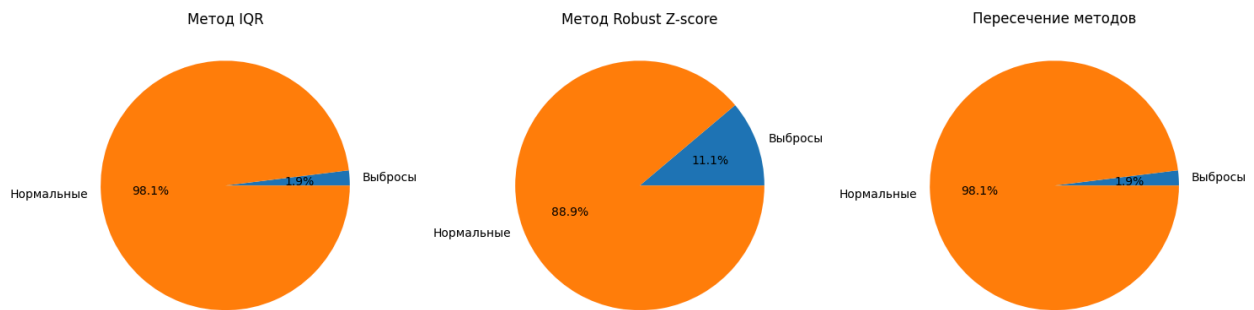
```

intersection_counts = [np.sum(intersection_outliers), len(intersection_outlier
axes[2].pie(intersection_counts, labels=['Выбросы', 'Нормальные'], autopct='%1
axes[2].set_title('Пересечение методов')

plt.tight_layout()
plt.show()

print(f"Объектов с выбросами (IQR): {np.sum(outliers_iqr)}")
print(f"Объектов с выбросами (Robust Z-score): {np.sum(outliers_robust)}")
print(f"Объектов с выбросами (пересечение): {np.sum(intersection_outliers)}")

```



Объектов с выбросами (IQR): 186
 Объектов с выбросами (Robust Z-score): 1063
 Объектов с выбросами (пересечение): 186

1.6.1.4. Удаление объектов с выбросами

Сохраняем индексы выбросов для последующего удаления и удаляем их

```

In [156... outlier_indices = X_train_knn[intersection_outliers].index
print(f"Индексы объектов с выбросами: {list(outlier_indices)[:20]}...")

X_train_no_outliers = X_train_knn.drop(index=outlier_indices).copy()
y_train_no_outliers = y_train.drop(index=outlier_indices).copy()

print(f"\nРазмеры данных после удаления выбросов:")
print(f"Обучающая выборка: {X_train_no_outliers.shape}")
print(f"Целевая переменная: {y_train_no_outliers.shape}")

```

Индексы объектов с выбросами: [620, 3068, 1745, 1889, 530, 782, 476, 2870, 809, 1880, 2177, 1034, 1259, 3077, 3329, 323, 2375, 3527, 611, 2114]...

Размеры данных после удаления выбросов:
 Обучающая выборка: (9361, 13)
 Целевая переменная: (9361,)

1.6.2. Поиск и удаление аномалий

1.6.2.1. Подготовка данных и снижение размерности

Масштабируем данные для алгоритмов поиска аномалий, также снижаем

размерность для визуализации

```
In [157... scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train_no_outliers.select_dtypes(include=[np.

print("Снижение размерности для визуализации...")
umap_reducer = UMAP(n_components=2, random_state=42)
X_2d = umap_reducer.fit_transform(X_scaled)

def draw_anomaly(X_2d, anomaly_mask):
    plt.figure(figsize=(8, 6))
    plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5, s=20, c='blue', label='Норм
    plt.scatter(X_2d[anomaly_mask, 0], X_2d[anomaly_mask, 1], c='red', marker=
    plt.xlabel('UMAP Component 1')
    plt.ylabel('UMAP Component 2')
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

def draw_anomaly_pie(anomaly_mask, title=""):
    normal_count = np.sum(~anomaly_mask)
    anomaly_count = np.sum(anomaly_mask)

    labels = ['Нормальные объекты', 'Аномалии']
    sizes = [normal_count, anomaly_count]

    plt.figure(figsize=(8, 6))
    plt.pie(sizes, labels=[f'{normal_count}\n({normal_count/len(anomaly_mask)*
    plt.title(f'{title}\nВсего объектов: {len(anomaly_mask)}', fontsize=14, pa
    plt.legend(labels, loc='lower right')
    plt.tight_layout()
    plt.show()
```

Снижение размерности для визуализации...

1.6.2.2. Применение Isolation Forest

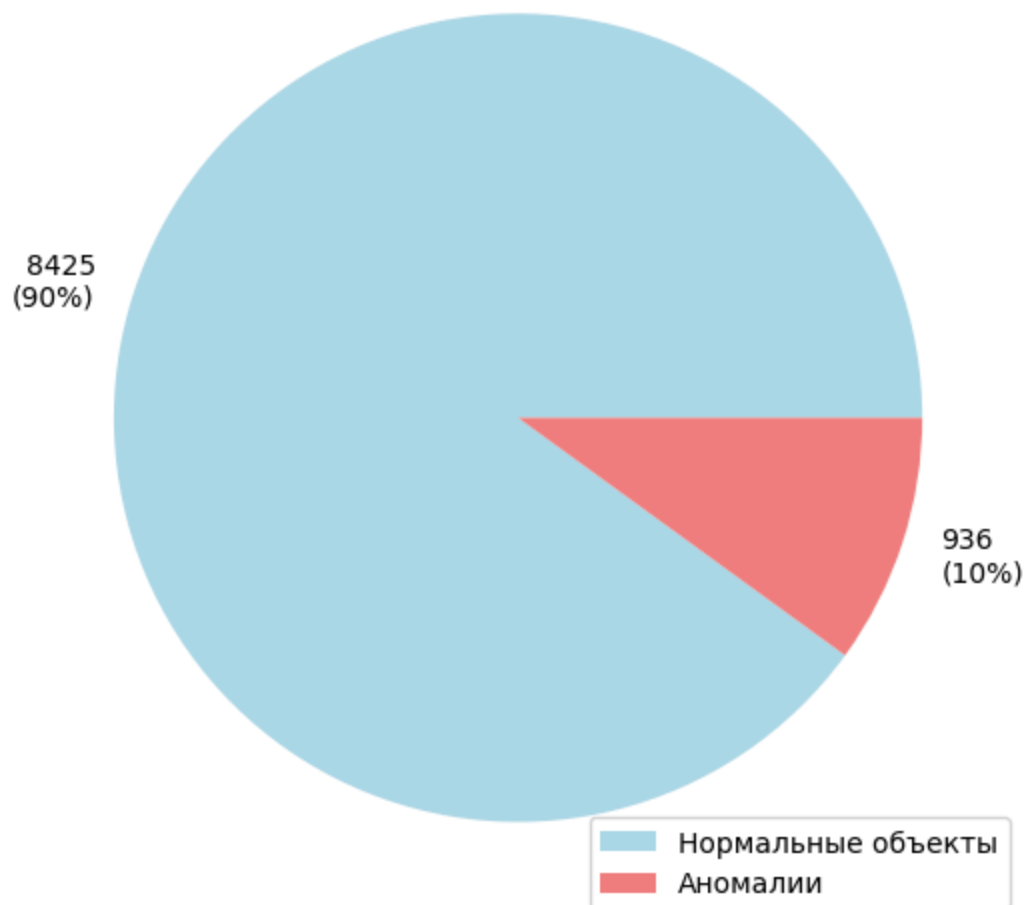
```
In [158... print("Применение Isolation Forest для поиска аномалий...")
iso_forest = IsolationForest(contamination=0.1, random_state=42)
iso_predictions = iso_forest.fit_predict(X_scaled)
iso_anomaly_mask = iso_predictions == -1

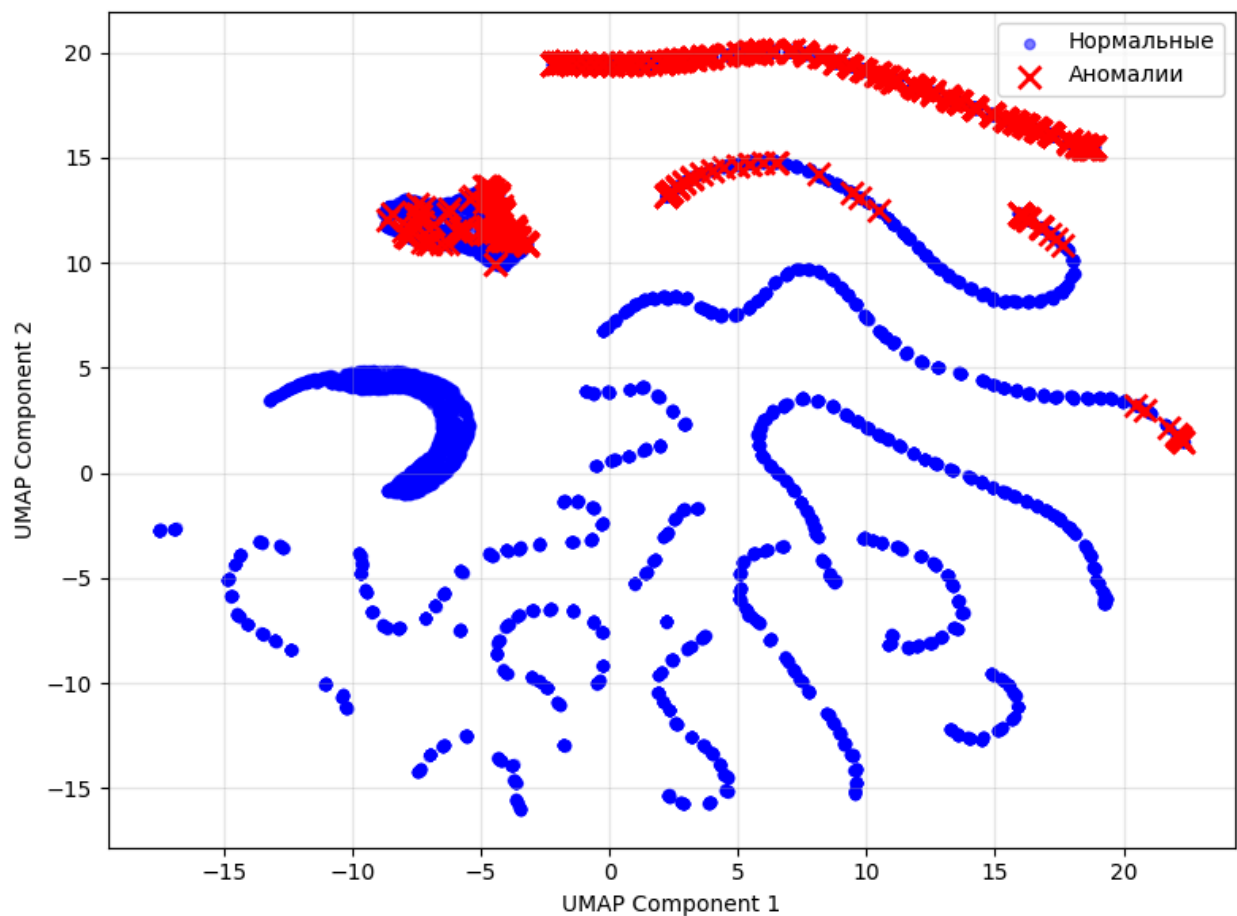
print(f"Isolation Forest обнаружено аномалий: {np.sum(iso_anomaly_mask)}")
draw_anomaly_pie(iso_anomaly_mask, "Isolation Forest")
draw_anomaly(X_2d, iso_anomaly_mask)
```

Применение Isolation Forest для поиска аномалий...

Isolation Forest обнаружено аномалий: 936

Isolation Forest
Всего объектов: 9361





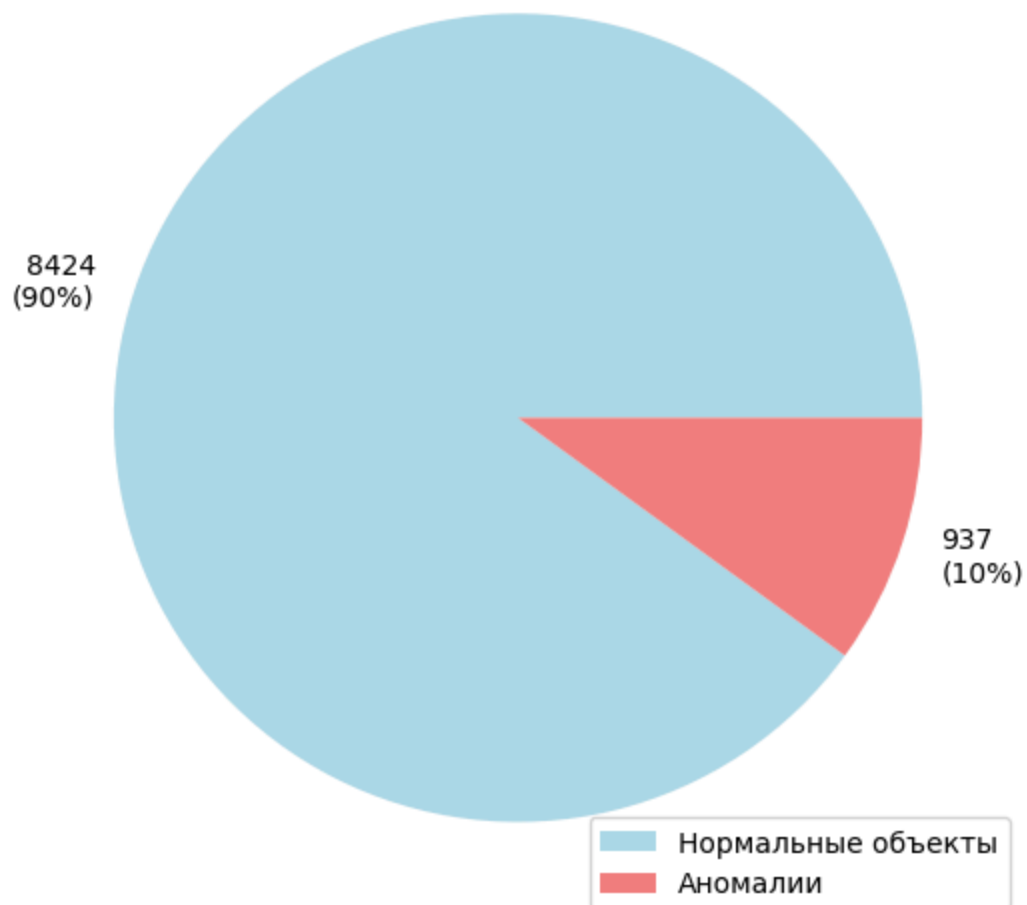
1.6.2.3. Применение One-Class SVM

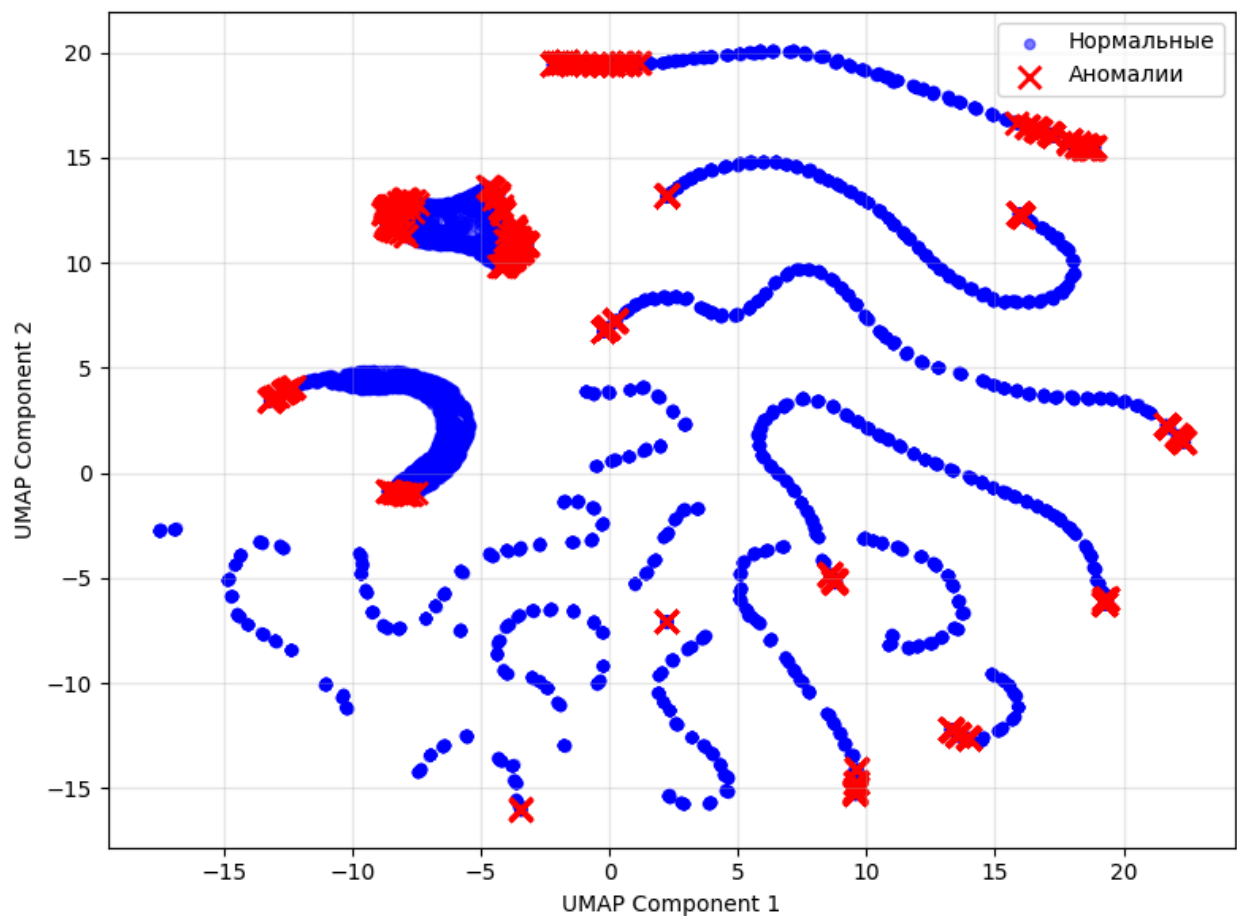
```
In [159... print("Применение One-Class SVM для поиска аномалий...")
oc_svm = OneClassSVM(nu=0.1, kernel='rbf', gamma='scale')
oc_svm_predictions = oc_svm.fit_predict(X_scaled)
oc_svm_anomaly_mask = oc_svm_predictions == -1

print(f"One-Class SVM обнаружено аномалий: {np.sum(oc_svm_anomaly_mask)}")
draw_anomaly_pie(oc_svm_anomaly_mask, "One-Class SVM")
draw_anomaly(X_2d, oc_svm_anomaly_mask)
```

Применение One-Class SVM для поиска аномалий...
 One-Class SVM обнаружено аномалий: 937

One-Class SVM
Всего объектов: 9361





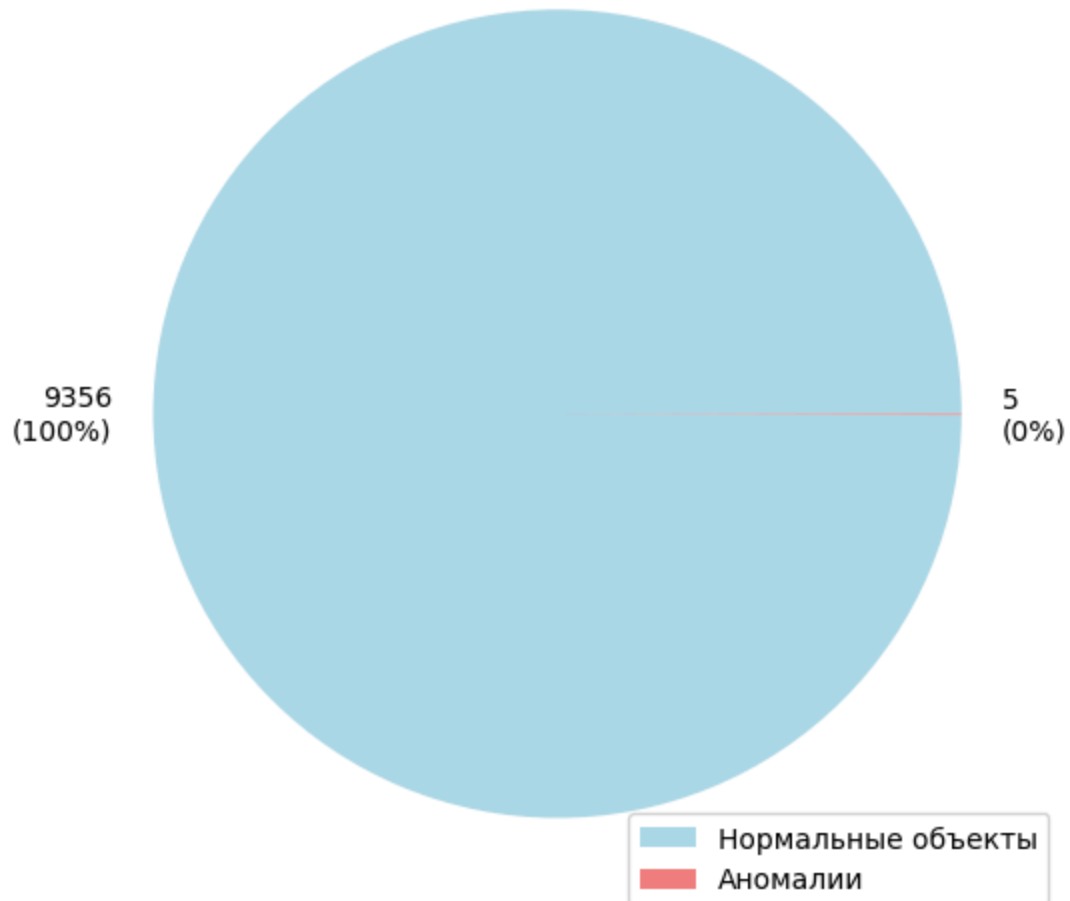
1.6.2.4. Применение DBSCAN

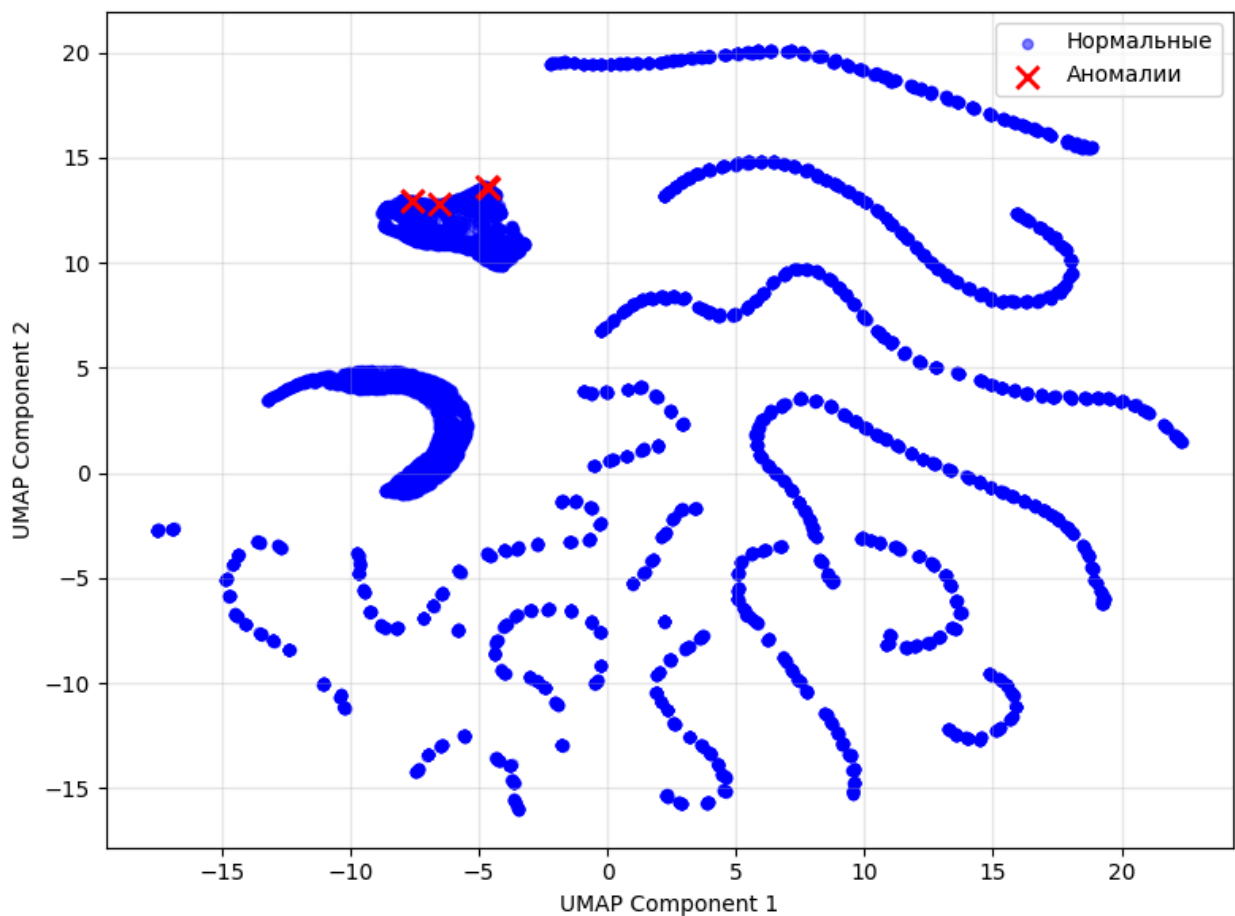
```
In [160... print("Применение DBSCAN для поиска аномалий...")
dbscan = DBSCAN(eps=0.3, min_samples=10)
dbscan_labels = dbscan.fit_predict(X_scaled)
dbscan_anomaly_mask = dbscan_labels == -1

print(f"DBSCAN обнаружено аномалий: {np.sum(dbscan_anomaly_mask)}")
draw_anomaly_pie(dbscan_anomaly_mask, "DBSCAN")
draw_anomaly(X_2d, dbscan_anomaly_mask)
```

Применение DBSCAN для поиска аномалий...
DBSCAN обнаружено аномалий: 5

DBSCAN
Всего объектов: 9361





1.6.2.5. Определение общих аномалий

Объединяем результаты всех трех методов

```
In [161]: anomaly_masks = np.array([iso_anomaly_mask, oc_svm_anomaly_mask, dbscan_anomal
common_anomaly_mask = np.all(anomaly_masks, axis=0)

print(f"Общие аномалии (все 3 метода): {np.sum(common_anomaly_mask)}")
draw_anomaly_pie(common_anomaly_mask, "Общие аномалии (3 метода)")
draw_anomaly(X_2d, common_anomaly_mask)

plt.figure(figsize=(15, 10))

# График 1: Isolation Forest
plt.subplot(2, 3, 1)
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5, s=10, c='blue')
plt.scatter(X_2d[iso_anomaly_mask, 0], X_2d[iso_anomaly_mask, 1], c='red', mar
plt.title(f'Isolation Forest\n({np.sum(iso_anomaly_mask)} аномалий)')
plt.xlabel('UMAP 1')
plt.ylabel('UMAP 2')
plt.grid(True, alpha=0.3)

# График 2: One-Class SVM
plt.subplot(2, 3, 2)
```

```

plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5, s=10, c='blue')
plt.scatter(X_2d[oc_svm_anomaly_mask, 0], X_2d[oc_svm_anomaly_mask, 1], c='red')
plt.title(f'One-Class SVM\n({np.sum(oc_svm_anomaly_mask)} аномалий)')
plt.xlabel('UMAP 1')
plt.ylabel('UMAP 2')
plt.grid(True, alpha=0.3)

# График 3: DBSCAN
plt.subplot(2, 3, 3)
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5, s=10, c='blue')
plt.scatter(X_2d[dbscan_anomaly_mask, 0], X_2d[dbscan_anomaly_mask, 1], c='red')
plt.title(f'DBSCAN\n({np.sum(dbscan_anomaly_mask)} аномалий)')
plt.xlabel('UMAP 1')
plt.ylabel('UMAP 2')
plt.grid(True, alpha=0.3)

# График 4: Общие аномалии
plt.subplot(2, 3, 4)
plt.scatter(X_2d[:, 0], X_2d[:, 1], alpha=0.5, s=10, c='blue')
plt.scatter(X_2d[common_anomaly_mask, 0], X_2d[common_anomaly_mask, 1], c='red')
plt.title(f'Общие аномалии (3 метода)\n({np.sum(common_anomaly_mask)} аномалий)')
plt.xlabel('UMAP 1')
plt.ylabel('UMAP 2')
plt.grid(True, alpha=0.3)

# График 5: Сравнение обнаруженных аномалий
plt.subplot(2, 3, 5)
all_methods = iso_anomaly_mask.astype(int) + oc_svm_anomaly_mask.astype(int) +
unique, counts = np.unique(all_methods, return_counts=True)

plt.bar(['0 методов', '1 метод', '2 метода', '3 метода'], counts)
plt.title('Количество методов, обнаруживших аномалии')
plt.ylabel('Количество объектов')
for i, v in enumerate(counts):
    plt.text(i, v + 0.1, str(v), ha='center')

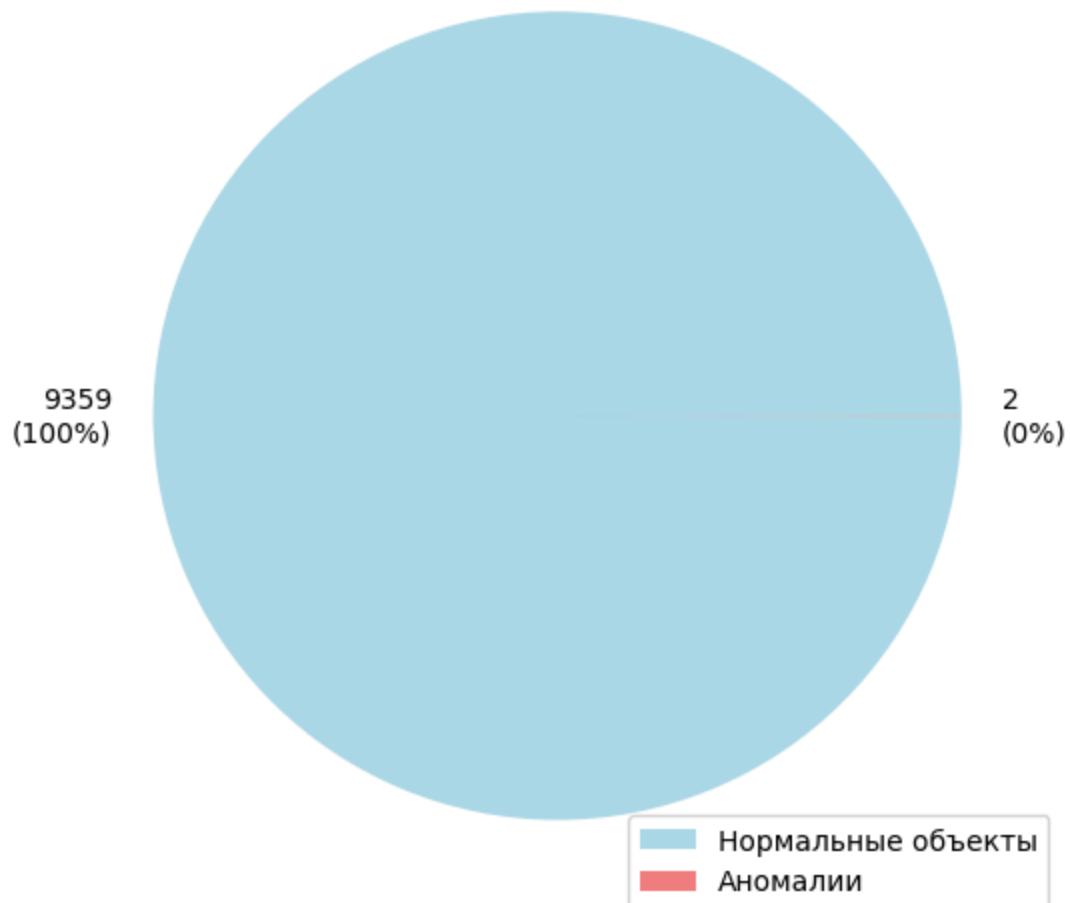
# График 6: Финал - очищенные данные
plt.subplot(2, 3, 6)
X_clean_2d = X_2d[~common_anomaly_mask]
plt.scatter(X_clean_2d[:, 0], X_clean_2d[:, 1], alpha=0.5, s=10, c='blue')
plt.title(f'Очищенные данные\n({len(X_clean_2d)} объектов)')
plt.xlabel('UMAP 1')
plt.ylabel('UMAP 2')
plt.grid(True, alpha=0.3)

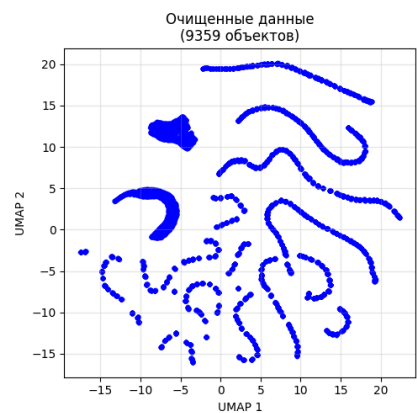
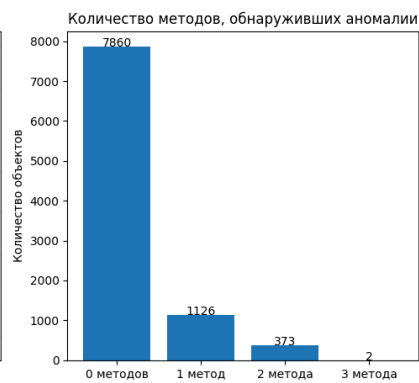
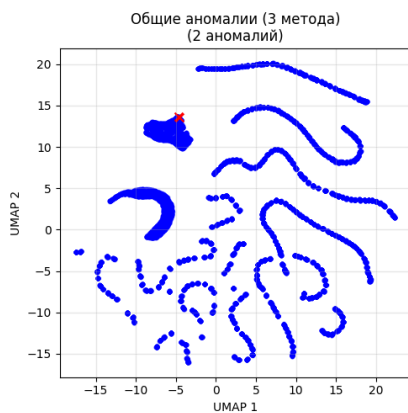
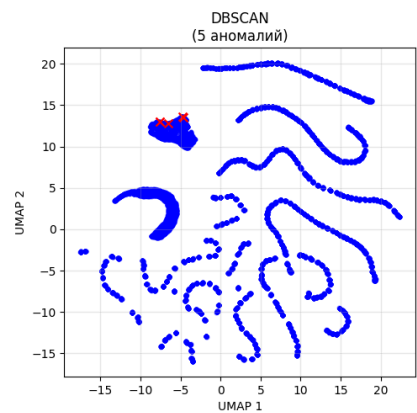
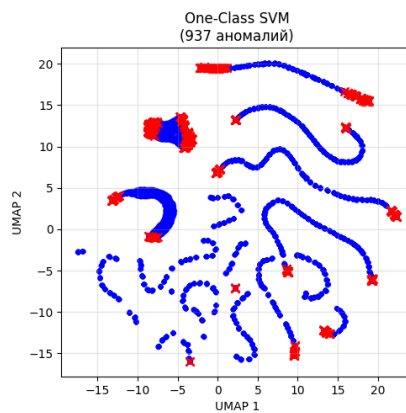
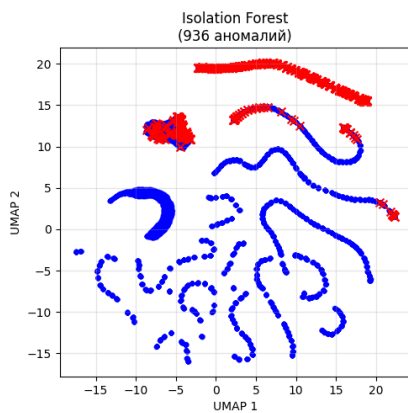
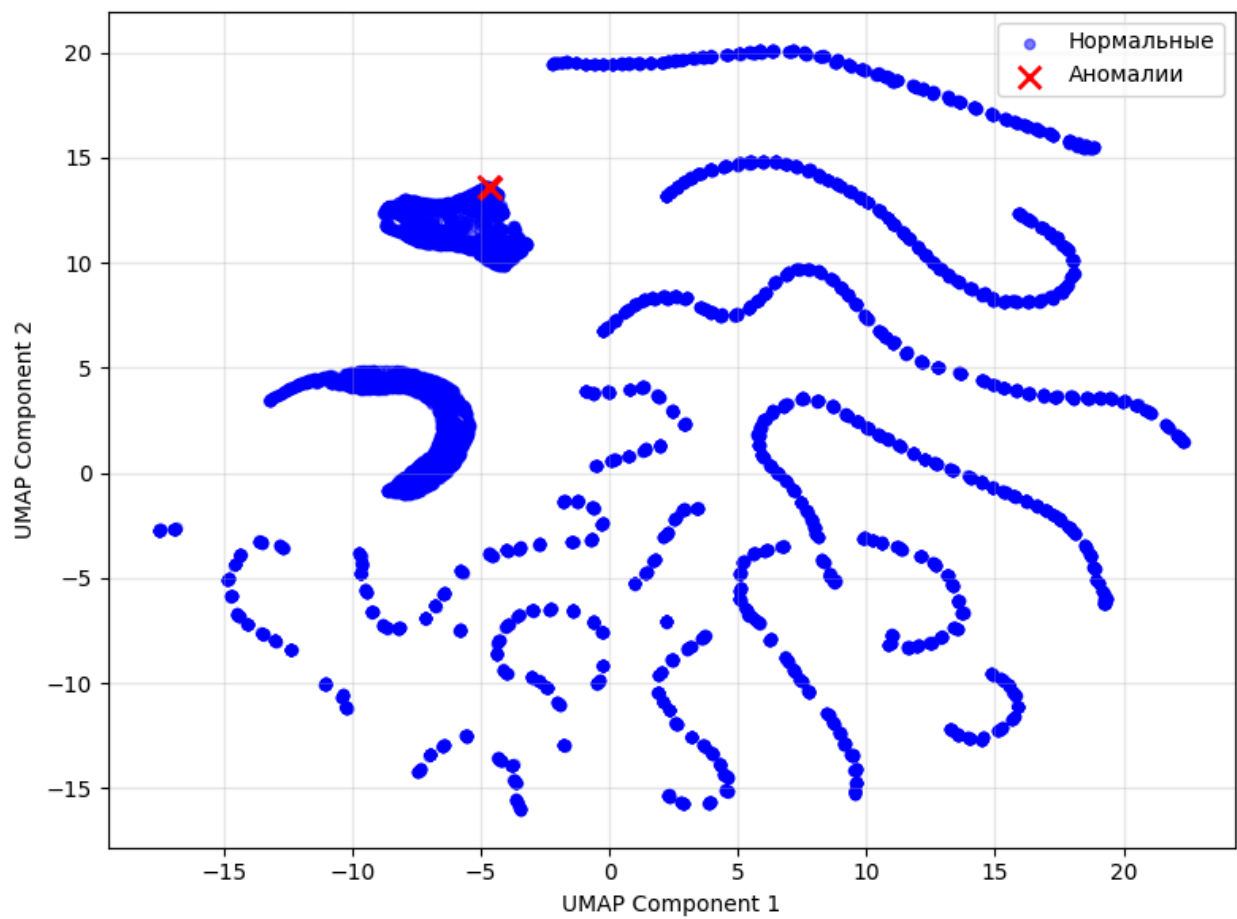
plt.tight_layout()
plt.show()

```

Общие аномалии (все 3 метода): 2

Общие аномалии (3 метода)
Всего объектов: 9361





1.6.2.6. Удаление аномалий и итоговая статистика

```
In [162... anomaly_indices = X_train_no_outliers.index[common_anomaly_mask]
X_train_clean = X_train_no_outliers.drop(index=anomaly_indices).copy()
y_train_clean = y_train_no_outliers.drop(index=anomaly_indices).copy()

initial_size = len(X_train_knn)
outliers_removed = len(outlier_indices)
anomalies_removed = len(anomaly_indices)
final_size = len(X_train_clean)

print("="*60)
print("ИТОГОВАЯ СТАТИСТИКА")
print("="*60)
print(f"{'Начальный размер обучающей выборки:':<40} {initial_size:>10}")
print(f"{'Удалено объектов с выбросами (IQR & Robust Z)':<40} {outliers_removed:>10}")
print(f"{'Удалено аномальных объектов (3 метода)':<40} {anomalies_removed:>10}")
print(f"{'Итоговый размер обучающей выборки:':<40} {final_size:>10}")
print(f"{'Всего удалено объектов:':<40} {initial_size - final_size:>10}")
print(f"{'Процент удаленных данных:':<40} {(initial_size - final_size)/initial_size:>10}")
print("="*60)

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

X_initial_2d = umap_reducer.transform(scaler.transform(X_train_knn.select_dtypes(include=[np.number])))
axes[0].scatter(X_initial_2d[:, 0], X_initial_2d[:, 1], alpha=0.5, s=10, c='blue')
axes[0].set_title(f'Исходные данные\n({initial_size} объектов)')
axes[0].set_xlabel('UMAP 1')
axes[0].set_ylabel('UMAP 2')
axes[0].grid(True, alpha=0.3)

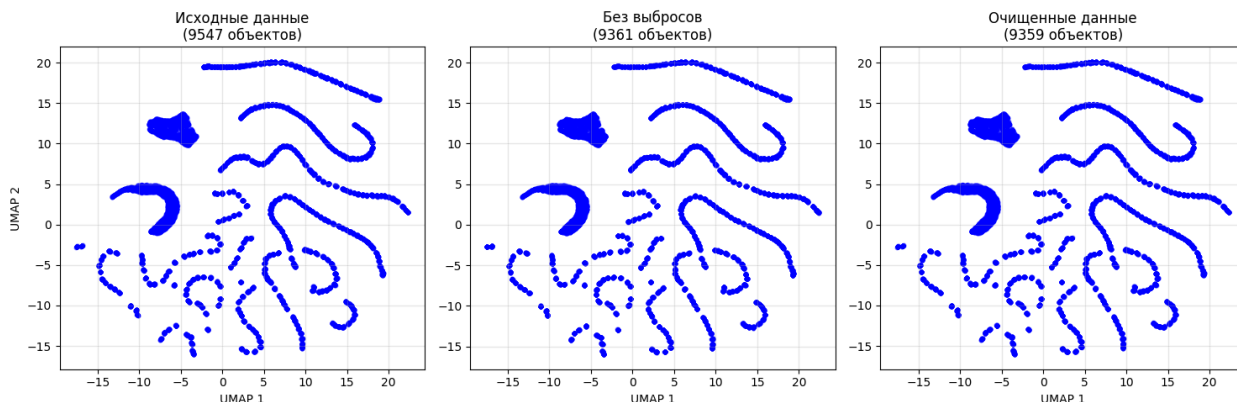
X_no_outliers_2d = umap_reducer.transform(scaler.transform(X_train_no_outliers.select_dtypes(include=[np.number])))
axes[1].scatter(X_no_outliers_2d[:, 0], X_no_outliers_2d[:, 1], alpha=0.5, s=10, c='blue')
axes[1].set_title(f'Без выбросов\n({len(X_train_no_outliers)} объектов)')
axes[1].set_xlabel('UMAP 1')
axes[1].grid(True, alpha=0.3)

X_clean_2d = umap_reducer.transform(scaler.transform(X_train_clean.select_dtypes(include=[np.number])))
axes[2].scatter(X_clean_2d[:, 0], X_clean_2d[:, 1], alpha=0.5, s=10, c='blue')
axes[2].set_title(f'Очищенные данные\n({final_size} объектов)')
axes[2].set_xlabel('UMAP 1')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

ИТОГОВАЯ СТАТИСТИКА

Начальный размер обучающей выборки:	9547	
Удалено объектов с выбросами (IQR & Robust Z):	186	
Удалено аномальных объектов (3 метода):	2	
Итоговый размер обучающей выборки:	9359	
Всего удалено объектов:	188	
Процент удаленных данных:	1.97%	



1.7. Подведение итогов раздела 1

Данные прошли полный цикл предобработки и готовы к моделированию.

Исходный набор (11934 строк, 20 признаков) был очищен от неинформативных столбцов: удалены уникальный идентификатор, дублирующие признаки, а также столбцы с более чем 80% пропусков. В результате осталось **14 признаков**, включая один категориальный (Тип Вектора Мощи).

Категориальный признак был успешно закодирован методом Ordinal Encoding с сохранением естественного порядка категорий.

Пропуски в основном сосредоточены в одном признаке («Скорость перехода через портал», ~10% пропусков). Для их заполнения были применены статистический подход (на основе KL-дивергенции выбрано значение 3.0866) и машинный подход (KNNImputer).

Выбросы обнаружены двумя методами (IQR и Robust Z-score). Их пересечение составило **186 объектов**, которые были удалены из обучающей выборки.

Аномалии выявлены с помощью трёх алгоритмов (Isolation Forest, One-Class SVM, DBSCAN). Общими для всех методов оказались **2 объекта**, которые также были исключены.

В итоге **обучающая выборка сократилась на 188 строк (1.97%)**, что

является приемлемым для сохранения репрезентативности данных.

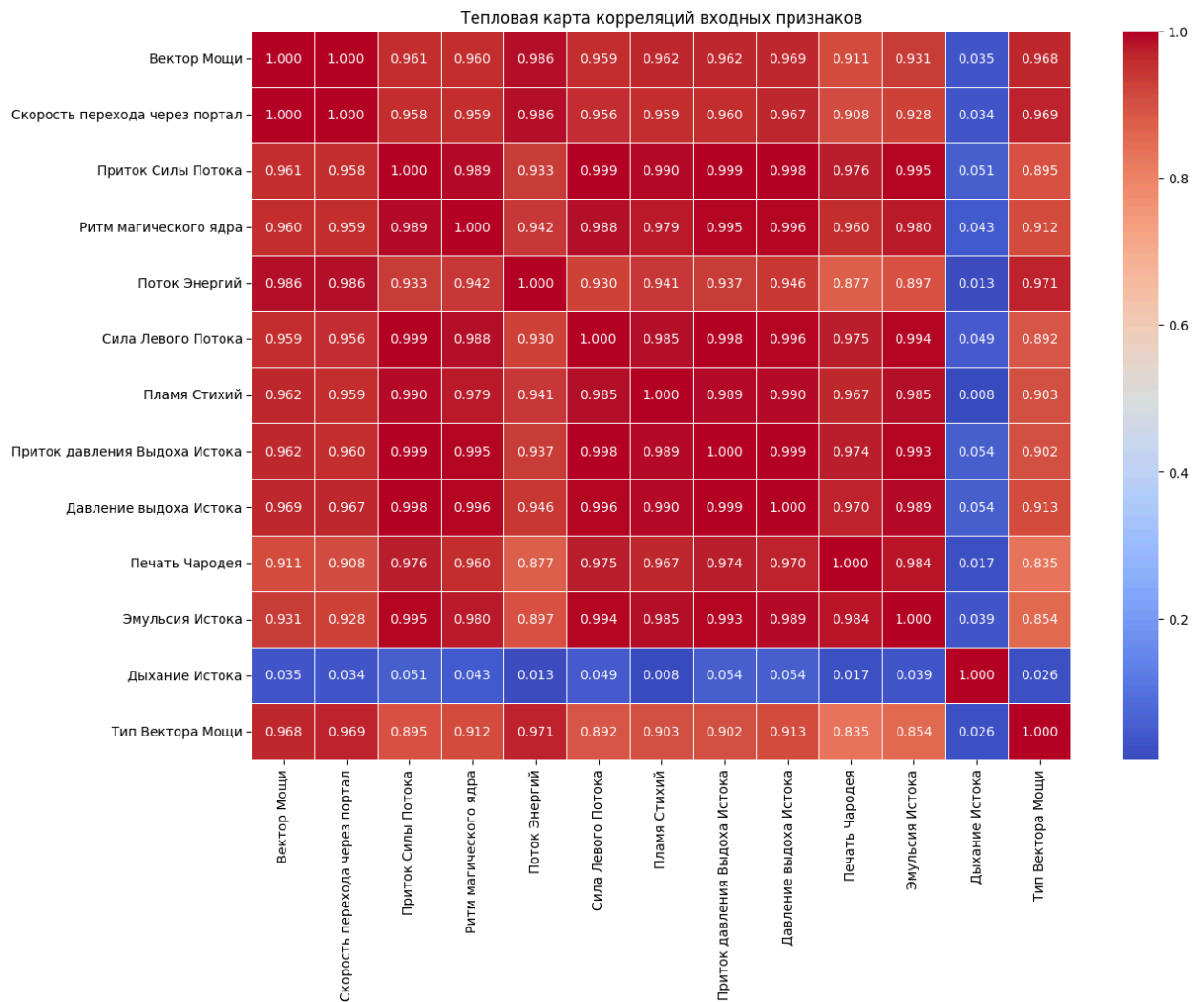
2. Генерация новых признаков

2.1. Корреляционный анализ входных признаков (построение тепловых карт корреляции)

```
In [163... plt.figure(figsize=(14, 10))
corr_matrix = X_train_clean.corr()
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm', linewidths=0.5)
plt.title('Тепловая карта корреляций входных признаков')
plt.show()

print("Анализ сильных корреляций (>0.99):")
strong_correlations = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_val = corr_matrix.iloc[i, j]
        if abs(corr_val) > 0.99:
            strong_correlations.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                corr_val
            ))

if strong_correlations:
    print(f" Найдено {len(strong_correlations)} сильных корреляций:")
    for col1, col2, corr_val in strong_correlations:
        print(f"    {col1} <-> {col2}: {corr_val:.4f}")
else:
    print(" Сильных корреляций (>0.99) не обнаружено")
```



Анализ сильных корреляций (>0.99):

Найдено 13 сильных корреляций:

Вектор Мощи <-> Скорость перехода через портал: 0.9999

Приток Силы Потока <-> Сила Левого Потока: 0.9991

Приток Силы Потока <-> Пламя Стихий: 0.9904

Приток Силы Потока <-> Приток давления Выдоха Истока: 0.9989

Приток Силы Потока <-> Давление выдоха Истока: 0.9975

Приток Силы Потока <-> Эмульсия Истока: 0.9951

Ритм магического ядра <-> Приток давления Выдоха Истока: 0.9949

Ритм магического ядра <-> Давление выдоха Истока: 0.9959

Сила Левого Потока <-> Приток давления Выдоха Истока: 0.9980

Сила Левого Потока <-> Давление выдоха Истока: 0.9961

Сила Левого Потока <-> Эмульсия Истока: 0.9941

Приток давления Выдоха Истока <-> Давление выдоха Истока: 0.9994

Приток давления Выдоха Истока <-> Эмульсия Истока: 0.9929

2.2. Исправление проблемы мультиколлинеарности в данных

Сгенерируем новые признаки по формулам прикладной области

```

In [164... X_train_transformed = X_train_clean.copy()
X_test_transformed = X_test_knn.copy()

X_train_transformed = X_train_transformed.drop(columns=['Вектор Мощи'])
X_test_transformed = X_test_transformed.drop(columns=['Вектор Мощи'])

X_train_transformed['Суммарная_мощность_потоков'] = X_train_transformed['Сила
X_test_transformed['Суммарная_мощность_потоков'] = X_test_transformed['Сила Ле

X_train_transformed['Общая_сила_ядра'] = X_train_transformed['Ритм магического
X_test_transformed['Общая_сила_ядра'] = X_test_transformed['Ритм магического я

X_train_transformed['Общее_давление_выхода'] = X_train_transformed['Приток дав
X_test_transformed['Общее_давление_выхода'] = X_test_transformed['Приток давле

X_train_transformed['Магическая_производительность'] = X_train_transformed['Ск
X_test_transformed['Магическая_производительность'] = X_test_transformed['Скор

X_train_transformed['Эффективность_ядра'] = X_train_transformed['Общая_сила_яд
X_test_transformed['Эффективность_ядра'] = X_test_transformed['Общая_сила_ядра

X_train_transformed['Магическая_мощность'] = X_train_transformed['Эмульсия Ист
X_test_transformed['Магическая_мощность'] = X_test_transformed['Эмульсия Исток

```

Удалим исходные признаки, которые были заменены

```

In [165... cols_to_drop = [
    'Скорость перехода через портал',
    'Приток Силы Потока',
    'Ритм магического ядра',
    'Приток давления Выдоха Истока',
    'Давление выдоха Истока',
    'Эмульсия Истока',
    'Сила Левого Потока',
    'Пламя Стихий'
]
X_train_transformed = X_train_transformed.drop(columns=cols_to_drop)
X_test_transformed = X_test_transformed.drop(columns=cols_to_drop)

print("Размерность после замены признаков:")
print(f"  Train: {X_train_transformed.shape}")
print(f"  Test: {X_test_transformed.shape}")

```

Размерность после замены признаков:

```

Train: (9359, 10)
Test: (2387, 10)

```

Проверка мультиколлинеарности после преобразований

```

In [166... plt.figure(figsize=(14, 10))
corr_matrix = X_train_transformed.corr()
sns.heatmap(corr_matrix, annot=True, fmt='.3f', cmap='coolwarm', linewidths=0.
plt.title('Тепловая карта корреляций входных признаков')

```

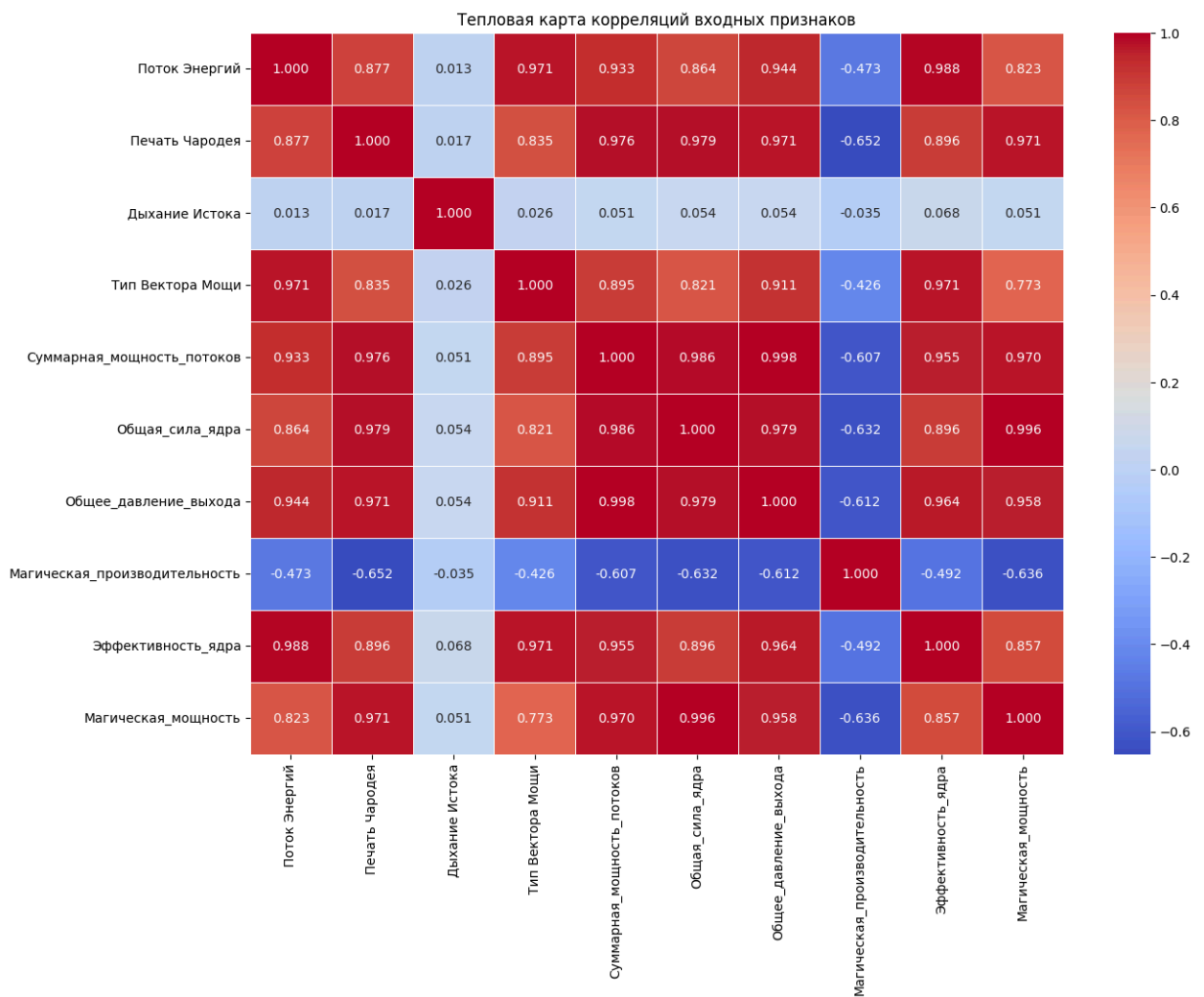
```

plt.show()

print("Анализ сильных корреляций (>0.99):")
strong_correlations = []
for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        corr_val = corr_matrix.iloc[i, j]
        if abs(corr_val) > 0.99:
            strong_correlations.append((
                corr_matrix.columns[i],
                corr_matrix.columns[j],
                corr_val
            ))

if strong_correlations:
    print(f" Найдено {len(strong_correlations)} сильных корреляций:")
    for coll, col2, corr_val in strong_correlations:
        print(f"      {coll} <-> {col2}: {corr_val:.4f}")
else:
    print(" Сильных корреляций (>0.99) не обнаружено")

```



Анализ сильных корреляций (>0.99):

Найдено 2 сильных корреляций:

Суммарная_мощность_потоков $\leftrightarrow$ Общее_давление_выхода: 0.9978

Общая_сила_ядра $\leftrightarrow$ Магическая_мощность: 0.9959

```
In [167... cols_to_drop_final = ['Общее_давление_выхода', 'Магическая_мощность']
X_train_transformed = X_train_transformed.drop(columns=cols_to_drop_final)
X_test_transformed = X_test_transformed.drop(columns=cols_to_drop_final)
```

2.3. Скалирование данных

Проведём анализ диспропорции

```
In [168... print("Статистики до скалирования:")
print(X_train_transformed.describe())

std_values = X_train_transformed.std()
print("\nСтандартные отклонения:")
print(std_values)
print(f"\nМаксимальное std: {std_values.max():.2f}")
print(f"Минимальное std: {std_values.min():.2f}")
print(f"Отношение max/min: {std_values.max()/std_values.min():.2f}")
```

Статистики до скалирования:

	Поток Энергий	Печать Чародея	Дыхание Истока	Тип Вектора Мощи \
count	9359.000000	9359.000000	9359.000000	9359.000000
mean	8165.934474	32.449774	0.975416	1.635111
std	1080.157126	24.781643	0.014643	1.153477
min	6589.002000	0.000000	0.950000	0.000000
25%	7048.821000	13.015500	0.963000	1.000000
50%	8477.011000	25.123000	0.976000	2.000000
75%	9129.052000	44.225000	0.988000	3.000000
max	9797.103000	90.689000	1.000000	3.000000

	Суммарная_мощность_потоков	Общая_сила_ядра \
count	9359.000000	9.359000e+03
mean	26724.938364	7.473934e+06
std	21814.781500	8.128504e+06
min	267.547000	3.556916e+04
25%	8496.047500	1.216267e+06
50%	21979.413000	4.358544e+06
75%	39661.043000	1.093640e+07
max	74075.284000	2.714001e+07

	Магическая_производительность	Эффективность_ядра
count	9359.000000	9.359000e+03
mean	13.958021	8.640862e+06
std	4.232819	4.363026e+06
min	4.566036	4.373923e+05
25%	10.472061	4.886796e+06
50%	13.677873	8.733970e+06
75%	17.945581	1.262845e+07
max	25.094634	1.592358e+07

Стандартные отклонения:

Поток Энергий	1.080157e+03
Печать Чародея	2.478164e+01
Дыхание Истока	1.464273e-02
Тип Вектора Мощи	1.153477e+00
Суммарная_мощность_потоков	2.181478e+04
Общая_сила_ядра	8.128504e+06
Магическая_производительность	4.232819e+00
Эффективность_ядра	4.363026e+06

dtype: float64

Максимальное std: 8128503.59

Минимальное std: 0.01

Отношение max/min: 555122254.12

Выполним три вида скалирования

```
In [169... X_train_original = X_train_transformed.copy()
X_test_original = X_test_transformed.copy()

scaler_standard = StandardScaler()
X_train_standard = pd.DataFrame(
```

```

    scaler_standard.fit_transform(X_train_transformed),
    columns=X_train_transformed.columns,
    index=X_train_transformed.index
)
X_test_standard = pd.DataFrame(
    scaler_standard.transform(X_test_transformed),
    columns=X_test_transformed.columns,
    index=X_test_transformed.index
)

scaler_minmax = MinMaxScaler()
X_train_minmax = pd.DataFrame(
    scaler_minmax.fit_transform(X_train_transformed),
    columns=X_train_transformed.columns,
    index=X_train_transformed.index
)
X_test_minmax = pd.DataFrame(
    scaler_minmax.transform(X_test_transformed),
    columns=X_test_transformed.columns,
    index=X_test_transformed.index
)

scaler_robust = RobustScaler()
X_train_robust = pd.DataFrame(
    scaler_robust.fit_transform(X_train_transformed),
    columns=X_train_transformed.columns,
    index=X_train_transformed.index
)
X_test_robust = pd.DataFrame(
    scaler_robust.transform(X_test_transformed),
    columns=X_test_transformed.columns,
    index=X_test_transformed.index
)

print("Размеры после скалирования (все одинаковые):")
print(f"    Оригинал:          {X_train_original.shape}")
print(f"    Стандартизация:    {X_train_standard.shape}")
print(f"    MinMax:            {X_train_minmax.shape}")
print(f"    Robust:            {X_train_robust.shape}")

```

Размеры после скалирования (все одинаковые):

```

Оригинал:          (9359, 8)
Стандартизация:   (9359, 8)
MinMax:           (9359, 8)
Robust:           (9359, 8)

```

Визуализация распределений после скалирования

```

In [170... fig, axes = plt.subplots(2, 2, figsize=(16, 12))

axes[0, 0].boxplot([X_train_original[col] for col in X_train_original.columns])
axes[0, 0].set_title('Оригинальные данные (без скалирования)')
axes[0, 0].tick_params(axis='x', rotation=45)
axes[0, 0].grid(True, alpha=0.3)

```

```

axes[0, 1].boxplot([X_train_standard[col] for col in X_train_standard.columns])
axes[0, 1].set_title('Стандартизация (Z-score)')
axes[0, 1].tick_params(axis='x', rotation=45)
axes[0, 1].grid(True, alpha=0.3)

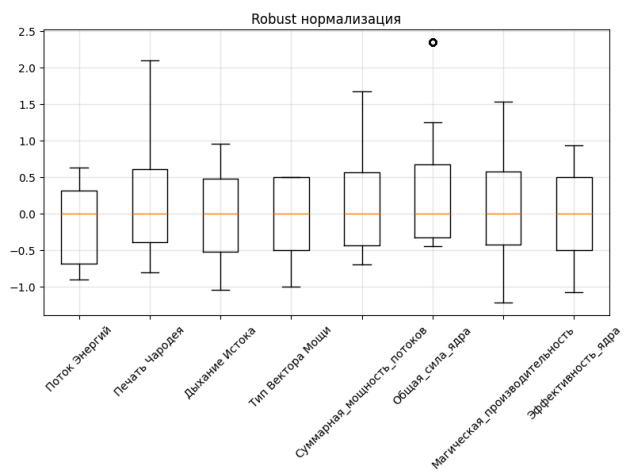
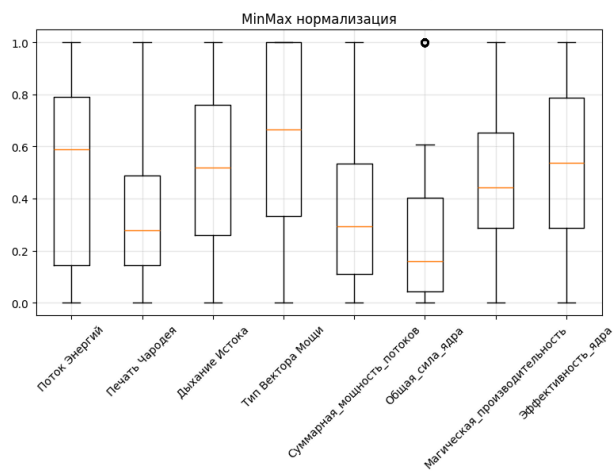
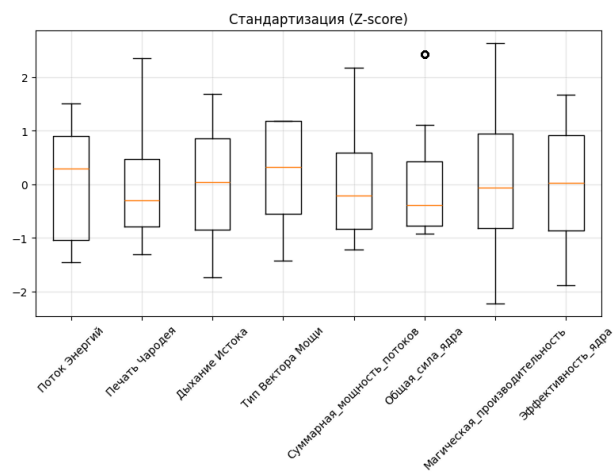
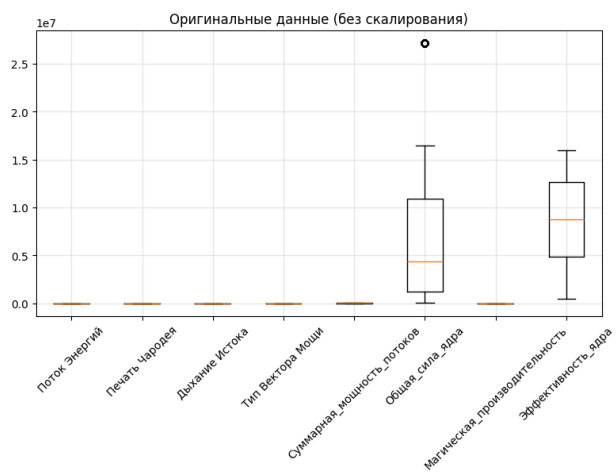
axes[1, 0].boxplot([X_train_minmax[col] for col in X_train_minmax.columns], la
axes[1, 0].set_title('MinMax нормализация')
axes[1, 0].tick_params(axis='x', rotation=45)
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].boxplot([X_train_robust[col] for col in X_train_robust.columns], la
axes[1, 1].set_title('Robust нормализация')
axes[1, 1].tick_params(axis='x', rotation=45)
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nСтандартные отклонения после разных видов скалирования:")
print("\n1. Стандартизация:")
print(X_train_standard.std())
print("\n2. MinMax нормализация:")
print(X_train_minmax.std())
print("\n3. Robust нормализация:")
print(X_train_robust.std())

```



Стандартные отклонения после разных видов скалирования:

1. Стандартизация:

Поток Энергий	1.000053
Печать Чародея	1.000053
Дыхание Истока	1.000053
Тип Вектора Мощи	1.000053
Суммарная_мощность_потоков	1.000053
Общая_сила_ядра	1.000053
Магическая_производительность	1.000053
Эффективность_ядра	1.000053
dtype: float64	

2. MinMax нормализация:

Поток Энергий	0.336697
Печать Чародея	0.273260
Дыхание Истока	0.292855
Тип Вектора Мощи	0.384492
Суммарная_мощность_потоков	0.295562
Общая_сила_ядра	0.299896
Магическая_производительность	0.206191
Эффективность_ядра	0.281737
dtype: float64	

3. Robust нормализация:

Поток Энергий	0.519249
Печать Чародея	0.794042
Дыхание Истока	0.585709
Тип Вектора Мощи	0.576738
Суммарная_мощность_потоков	0.699977
Общая_сила_ядра	0.836255
Магическая_производительность	0.566376
Эффективность_ядра	0.563578
dtype: float64	

2.4. Подведение итогов раздела 2

Проблема мультиколлинеарности была решена путём генерации новых признаков на основе формул прикладной области. Вместо сильно коррелирующих исходных признаков введены агрегированные показатели, такие как «Суммарная мощность потоков», «Общая сила ядра», «Магическая производительность» и другие. После преобразования количество сильных корреляций (>0.99) сократилось с 13 до 2.

Масштабирование признаков выполнено тремя способами (StandardScaler, MinMaxScaler, RobustScaler), что позволит в дальнейшем корректно сравнивать модели, чувствительные к масштабу данных.

3. Выбор моделей ML и метрик

Были выбраны следующие модели:

1. **LinearRegression** - Базовая линейная регрессия по методу наименьших квадратов. Не имеет гиперпараметров, крайне быстро обучается и даёт интерпретируемые коэффициенты.
2. **Ridge (гребневая регрессия)** - Линейная регрессия с L2-регуляризацией. Добавляет штраф за величину коэффициентов (α), что стабилизирует решение при мультиколлинеарности и избыточном признаках.
3. **Lasso (лассо-регрессия)** - Линейная регрессия с L1-регуляризацией. За счёт обнуления незначимых коэффициентов выполняет автоматический отбор признаков. Полезна для выявления действительно важных предикторов и борьбы с переобучением.
4. **RandomForestRegressor** - Ансамбль из сотен решающих деревьев, построенных на случайных подвыборках данных и признаков. Устойчив к шуму и выбросам, не требует масштабирования, автоматически учитывает нелинейности и взаимодействия признаков.
5. **GradientBoostingRegressor** - Классический градиентный бустинг из scikit-learn. Строит деревья последовательно: каждое следующее исправляет ошибки предыдущих. Имеет встроенную регуляризацию, хорошо работает с гетерогенными данными, но чувствителен к настройке гиперпараметров.
6. **SVR (Support Vector Regression)** - Метод опорных векторов для регрессии. Строит гиперплоскость в пространстве высокой размерности с максимальным отступом от точек. Особенно эффективен при небольшом и среднем объёме данных, требует тщательного масштабирования и подбора ядра (linear, rbf, poly).

```
In [171]: models = [  
    ('LinearRegression', LinearRegression),  
    ('Ridge', Ridge),  
    ('Lasso', Lasso),  
    ('RandomForestRegressor', RandomForestRegressor),  
    ('GradientBoostingRegressor', GradientBoostingRegressor),
```

```
( 'SVR', SVR)
]
```

Выбор пула метрик

1. **Mean Absolute Error (MAE)** — Среднее абсолютное отклонение предсказаний от истинных значений.
2. **Mean Squared Error (MSE)** — Среднее значение квадратов ошибок. Сильнее штрафует крупные отклонения.
3. **Коэффициент детерминации (R^2)** — Доля объяснённой дисперсии целевой переменной.
4. **Root Mean Squared Error (RMSE)** — Корень из MSE, более интерпретируемая метрика в исходных единицах целевой переменной.
5. **Mean Absolute Percentage Error (MAPE)** — Средняя абсолютная процентная ошибка, хорошо интерпретируется в бизнес-контексте.

```
In [172... def rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

metrics = {
    "MAE": mean_absolute_error,
    "MSE": mean_squared_error,
    "RMSE": rmse,
    "R2": r2_score,
    "MAPE": mean_absolute_percentage_error
}
```

Для дальнейшего подбора гиперпараметров была сформирована сетка:

```
In [173... model_grids = {
    'LinearRegression': {},
    'Ridge': {
        'alpha': [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
    },
    'Lasso': {
        'alpha': [0.001, 0.01, 0.1, 1.0, 10.0],
        'max_iter': [1000, 5000, 10000]
    },
    'RandomForestRegressor': {
        'n_estimators': [50, 100, 200, 300],
        'max_depth': [None, 10, 20, 30],
        'min_samples_split': [2, 5, 10],
        'min_samples_leaf': [1, 2, 4],
        'max_features': ['sqrt', 'log2', None]
    },
    'GradientBoostingRegressor': {
        'n_estimators': [50, 100, 200],
```



```

        'learning_rate': [0.001, 0.01, 0.1, 0.2],
        'max_depth': [3, 5, 7],
        'min_samples_split': [2, 5, 10],
        'subsample': [0.8, 0.9, 1.0]
    },
    'SVR': {
        'C': [0.1, 1.0, 10.0, 100.0],
        'epsilon': [0.01, 0.1, 0.5],
        'kernel': ['linear', 'rbf', 'poly']
    }
}

```

4. Обучение моделей ML и подбор гиперпараметров

Подбор и оценка будет выполняться на нескольких версиях датасета:

```

In [177... datasets = {
    'clean': X_train_clean, # До устранения мультиколлинеарности
    'transformed': X_train_transformed, # До скалирования
    'standard': X_train_standard, # Стандартизованные
    'minmax': X_train_minmax, # MinMax нормализованные
    'robust': X_train_robust # Robust нормализованные
}

test_datasets = {
    'clean': X_test_knn,
    'transformed': X_test_transformed,
    'standard': X_test_standard,
    'minmax': X_test_minmax,
    'robust': X_test_robust
}

```

Подбор гиперпараметров с помощью optuna

```

In [178... best_models = {}
best_params = {}
results = []

def create_objective(model_class, param_grid, X_train, y_train, cv_folds=3):
    def objective(trial):
        if model_class.__name__ == 'LinearRegression':
            params = {}
            model = model_class(**params)

        elif model_class.__name__ == 'Ridge':
            alpha = trial.suggest_float('alpha', 0.001, 100.0, log=True)
            model = model_class(alpha=alpha, random_state=42)

        elif model_class.__name__ == 'Lasso':

```

```

        alpha = trial.suggest_float('alpha', 0.001, 10.0, log=True)
        max_iter = trial.suggest_categorical('max_iter', [1000, 5000, 10000])
        model = model_class(alpha=alpha, max_iter=max_iter, random_state=42)

    elif model_class.__name__ == 'RandomForestRegressor':
        params = {
            'n_estimators': trial.suggest_int('n_estimators', 50, 300),
            'max_depth': trial.suggest_int('max_depth', 3, 30),
            'min_samples_split': trial.suggest_int('min_samples_split', 2, 5),
            'min_samples_leaf': trial.suggest_int('min_samples_leaf', 1, 5),
            'max_features': trial.suggest_categorical('max_features', ['sqrt', 'log2', 'best']),
            'random_state': 42,
            'n_jobs': -1
        }
        model = model_class(**params)

    elif model_class.__name__ == 'GradientBoostingRegressor':
        params = {
            'n_estimators': trial.suggest_int('n_estimators', 50, 200),
            'learning_rate': trial.suggest_float('learning_rate', 0.001, 0.1, log=True),
            'max_depth': trial.suggest_int('max_depth', 3, 7),
            'min_samples_split': trial.suggest_int('min_samples_split', 2, 5),
            'subsample': trial.suggest_float('subsample', 0.7, 1.0),
            'random_state': 42
        }
        model = model_class(**params)

    elif model_class.__name__ == 'SVR':
        params = {
            'C': trial.suggest_float('C', 0.1, 100.0, log=True),
            'epsilon': trial.suggest_float('epsilon', 0.01, 0.5),
            'kernel': trial.suggest_categorical('kernel', ['linear', 'rbf', 'poly']),
            'max_iter': 10000
        }
        model = model_class(**params)

    cv = KFold(n_splits=cv_folds, shuffle=True, random_state=42)
    scores = cross_val_score(model, X_train, y_train, cv=cv,
                              scoring='neg_mean_squared_error', n_jobs=-1)

    return scores.mean()

return objective

for dataset_name, X_train_data in datasets.items():
    print(f"\n{'='*60}")
    print(f"Подбор гиперпараметров для датасета: {dataset_name}")
    print(f"{'='*60}")

    best_models[dataset_name] = {}
    best_params[dataset_name] = {}

    for model_name, model_class in models:

```

```

print(f"\nМодель: {model_name}")

objective_func = create_objective(model_class, model_grids.get(model_name))

study = optuna.create_study(direction='maximize')

n_trials = 15

study.optimize(objective_func, n_trials=n_trials, show_progress_bar=True)

best_params[dataset_name][model_name] = study.best_params

if model_class.__name__ == 'LinearRegression':
    best_model = model_class()
else:
    if model_class.__name__ in ['RandomForestRegressor', 'GradientBoostingRegressor']:
        study.best_params['random_state'] = 42
        if model_class.__name__ == 'RandomForestRegressor':
            study.best_params['n_jobs'] = -1

    best_model = model_class(**study.best_params)

best_model.fit(X_train_data, y_train_clean)
best_models[dataset_name][model_name] = best_model

print(f"    Лучшие параметры: {study.best_params}")
print(f"    Лучшее значение MSE (CV): {-study.best_value:.4f}")

y_train_pred = best_model.predict(X_train_data)
train_mse = mean_squared_error(y_train_clean, y_train_pred)
train_mae = mean_absolute_error(y_train_clean, y_train_pred)
train_r2 = r2_score(y_train_clean, y_train_pred)

X_test_data = test_datasets[dataset_name]
y_test_pred = best_model.predict(X_test_data)
test_mse = mean_squared_error(y_test, y_test_pred)
test_mae = mean_absolute_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)

results.append({
    'Датасет': dataset_name,
    'Модель': model_name,
    'Параметры': str(study.best_params),
    'Train MSE': train_mse,
    'Train MAE': train_mae,
    'Train R2': train_r2,
    'Test MSE': test_mse,
    'Test MAE': test_mae,
    'Test R2': test_r2,
    'Разница MSE (Test-Train)': test_mse - train_mse,
    'Разница R2 (Train-Test)': train_r2 - test_r2
})

```

=====

Подбор гиперпараметров для датасета: clean

=====

Модель: LinearRegression

0%| | 0/15 [00:00<?, ?it/s]
Best trial: 0. Best value: -3.63725e-06: 100%|██████████| 15/15 [00:00<00:00, 36.23it/s]
Лучшие параметры: {}
Лучшее значение MSE (CV): 0.0000

Модель: Ridge

Best trial: 10. Best value: -3.63749e-06: 100%|██████████| 15/15 [00:00<00:00, 38.66it/s]
Лучшие параметры: {'alpha': 0.0013805096245154817}
Лучшее значение MSE (CV): 0.0000

Модель: Lasso

Best trial: 11. Best value: -3.8856e-05: 100%|██████████| 15/15 [00:01<00:00, 10.19it/s]
Лучшие параметры: {'alpha': 0.0010290986723969203, 'max_iter': 5000}
Лучшее значение MSE (CV): 0.0000

Модель: RandomForestRegressor

Best trial: 14. Best value: -6.31943e-07: 100%|██████████| 15/15 [00:26<00:00, 1.77s/it]
Лучшие параметры: {'n_estimators': 216, 'max_depth': 30, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': None}
Лучшее значение MSE (CV): 0.0000

Модель: GradientBoostingRegressor

Best trial: 13. Best value: -5.09153e-07: 100%|██████████| 15/15 [00:54<00:00, 3.61s/it]
Лучшие параметры: {'n_estimators': 176, 'learning_rate': 0.09565253076612736, 'max_depth': 7, 'min_samples_split': 4, 'subsample': 0.8389441192865098}
Лучшее значение MSE (CV): 0.0000

Модель: SVR

Best trial: 1. Best value: -5.60031e-05: 100%|██████████| 15/15 [00:00<00:00, 38.12it/s]
Лучшие параметры: {'C': 1.27809573512762, 'epsilon': 0.4626625878692497, 'kernel': 'linear'}
Лучшее значение MSE (CV): 0.0001

=====

Подбор гиперпараметров для датасета: transformed

=====

Модель: LinearRegression

Best trial: 0. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 67.21it/s]

Лучшие параметры: {}
Лучшее значение MSE (CV): 0.0000

Модель: Ridge

Best trial: 13. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 68.03it/s]

Лучшие параметры: {'alpha': 0.0010067503607405057}
Лучшее значение MSE (CV): 0.0000

Модель: Lasso

Best trial: 2. Best value: -4.99449e-05: 100%|██████████| 15/15 [00:02<00:00, 7.40it/s]

Лучшие параметры: {'alpha': 0.003209401203839267, 'max_iter': 10000}
Лучшее значение MSE (CV): 0.0000

Модель: RandomForestRegressor

Best trial: 14. Best value: -1.15595e-06: 100%|██████████| 15/15 [00:20<00:00, 1.34s/it]

Лучшие параметры: {'n_estimators': 256, 'max_depth': 20, 'min_samples_split': 7, 'min_samples_leaf': 3, 'max_features': None}
Лучшее значение MSE (CV): 0.0000

Модель: GradientBoostingRegressor

Best trial: 13. Best value: -7.57775e-07: 100%|██████████| 15/15 [00:35<00:00, 2.38s/it]

Лучшие параметры: {'n_estimators': 162, 'learning_rate': 0.12736195127383557, 'max_depth': 7, 'min_samples_split': 10, 'subsample': 0.7498428233218042}
Лучшее значение MSE (CV): 0.0000

Модель: SVR

Best trial: 2. Best value: -5.60031e-05: 100%|██████████| 15/15 [00:00<00:00, 43.36it/s]

Лучшие параметры: {'C': 1.238578240025105, 'epsilon': 0.12032458476127535, 'kernel': 'rbf'}
Лучшее значение MSE (CV): 0.0001

=====
Подбор гиперпараметров для датасета: standard
=====

Модель: LinearRegression

Best trial: 0. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 67.41it/s]

Лучшие параметры: {}
Лучшее значение MSE (CV): 0.0000

Модель: Ridge

Best trial: 12. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 65.55it/s]

Лучшие параметры: {'alpha': 0.00100170591787598}
Лучшее значение MSE (CV): 0.0000

Модель: Lasso

Best trial: 0. Best value: -5.5998e-05: 100%|██████████| 15/15 [00:00<00:00, 63.41it/s]

Лучшие параметры: {'alpha': 0.0026592597420705046, 'max_iter': 10000}
Лучшее значение MSE (CV): 0.0001

Модель: RandomForestRegressor

Best trial: 5. Best value: -9.02661e-07: 100%|██████████| 15/15 [00:21<00:00, 1.41s/it]

Лучшие параметры: {'n_estimators': 241, 'max_depth': 27, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': None}
Лучшее значение MSE (CV): 0.0000

Модель: GradientBoostingRegressor

Best trial: 10. Best value: -9.00829e-07: 100%|██████████| 15/15 [00:42<00:00, 2.85s/it]

Лучшие параметры: {'n_estimators': 199, 'learning_rate': 0.230801802066831, 'max_depth': 7, 'min_samples_split': 9, 'subsample': 0.9972386229396212}
Лучшее значение MSE (CV): 0.0000

Модель: SVR

Best trial: 0. Best value: -5.60031e-05: 100%|██████████| 15/15 [00:00<00:00, 60.72it/s]

Лучшие параметры: {'C': 29.239645750053803, 'epsilon': 0.38117758375782573, 'kernel': 'linear'}
Лучшее значение MSE (CV): 0.0001

=====
Подбор гиперпараметров для датасета: minmax
=====

Модель: LinearRegression

Best trial: 0. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 72.34it/s]

Лучшие параметры: {}
Лучшее значение MSE (CV): 0.0000

Модель: Ridge

Best trial: 13. Best value: -4.49883e-05: 100%|██████████| 15/15 [00:00<00:00, 65.07it/s]

Лучшие параметры: {'alpha': 0.0010395584092550698}
Лучшее значение MSE (CV): 0.0000

Модель: Lasso

Best trial: 0. Best value: -5.5998e-05: 100%|██████████| 15/15 [00:00<00:00, 67.88it/s]

Лучшие параметры: {'alpha': 0.2679897351360583, 'max_iter': 1000}
Лучшее значение MSE (CV): 0.0001

Модель: RandomForestRegressor

Best trial: 0. Best value: -9.69673e-07: 100%|██████████| 15/15 [00:23<00:00, 1.57s/it]

Лучшие параметры: {'n_estimators': 169, 'max_depth': 20, 'min_samples_split': 4, 'min_samples_leaf': 2, 'max_features': None}
Лучшее значение MSE (CV): 0.0000

Модель: GradientBoostingRegressor

Best trial: 14. Best value: -9.46283e-07: 100%|██████████| 15/15 [00:35<00:00, 2.34s/it]

Лучшие параметры: {'n_estimators': 199, 'learning_rate': 0.2678497984119415, 'max_depth': 7, 'min_samples_split': 9, 'subsample': 0.8844561555940917}
Лучшее значение MSE (CV): 0.0000

Модель: SVR

Best trial: 0. Best value: -5.60031e-05: 100%|██████████| 15/15 [00:00<00:00, 63.08it/s]

Лучшие параметры: {'C': 4.89043957695973, 'epsilon': 0.28783728723979574, 'kernel': 'rbf'}
Лучшее значение MSE (CV): 0.0001

=====
Подбор гиперпараметров для датасета: robust
=====

Модель: LinearRegression

Best trial: 0. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 67.99it/s]

Лучшие параметры: {}
Лучшее значение MSE (CV): 0.0000

Модель: Ridge

Best trial: 12. Best value: -4.49876e-05: 100%|██████████| 15/15 [00:00<00:00, 68.60it/s]

Лучшие параметры: {'alpha': 0.0010185422189375858}
Лучшее значение MSE (CV): 0.0000

Модель: Lasso

Best trial: 0. Best value: -5.5998e-05: 100%|██████████| 15/15 [00:00<00:00, 67.69it/s]

Лучшие параметры: {'alpha': 0.8387436344960194, 'max_iter': 1000}
Лучшее значение MSE (CV): 0.0001

Модель: RandomForestRegressor

Best trial: 14. Best value: -9.52752e-07: 100%|██████████| 15/15 [00:24<00:00, 1.62s/it]

Лучшие параметры: {'n_estimators': 207, 'max_depth': 30, 'min_samples_split': 6, 'min_samples_leaf': 1, 'max_features': None}
Лучшее значение MSE (CV): 0.0000

Модель: GradientBoostingRegressor

Best trial: 1. Best value: -9.36237e-07: 100%|██████████| 15/15 [00:35<00:00, 2.37s/it]

Лучшие параметры: {'n_estimators': 142, 'learning_rate': 0.07121981199132926, 'max_depth': 7, 'min_samples_split': 6, 'subsample': 0.7921492472257553}
Лучшее значение MSE (CV): 0.0000

Модель: SVR

Best trial: 1. Best value: -5.60031e-05: 100%|██████████| 15/15 [00:00<00:00, 59.80it/s]

Лучшие параметры: {'C': 9.407383763906024, 'epsilon': 0.3899718592767062, 'kernel': 'linear'}
Лучшее значение MSE (CV): 0.0001

5. Оценка моделей и выбор лучшей

```
In [179... results_df = pd.DataFrame(results)

results_df_sorted = results_df.sort_values(['Датасет', 'Test MSE'])

print("="*80)
print("СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ ВСЕХ МОДЕЛЕЙ НА ТЕСТОВОЙ ВЫБОРКЕ")
print("="*80)
print(results_df_sorted.to_string(index=False))

fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Сравнение качества моделей на разных датасетах', fontsize=16)

plt.subplot(2, 2, 1)
mse_pivot = results_df.pivot_table(index='Модель', columns='Датасет', values='Test MSE')
sns.heatmap(mse_pivot, annot=True, fmt='.3f', cmap='RdYlGn_r', ax=axes[0, 0])
axes[0, 0].set_title('Test MSE (чем меньше, тем лучше)')
axes[0, 0].tick_params(axis='x', rotation=45)

plt.subplot(2, 2, 2)
r2_pivot = results_df.pivot_table(index='Модель', columns='Датасет', values='Test R^2')
sns.heatmap(r2_pivot, annot=True, fmt='.3f', cmap='RdYlGn', ax=axes[0, 1])
axes[0, 1].set_title('Test R^2 (чем больше, тем лучше)')
axes[0, 1].tick_params(axis='x', rotation=45)

plt.subplot(2, 2, 3)
diff_pivot = results_df.pivot_table(index='Модель', columns='Датасет', values='Test Diff')
sns.heatmap(diff_pivot, annot=True, fmt='.3f', cmap='coolwarm', center=0, ax=axes[1, 0])
axes[1, 0].set_title('Разница MSE (Test - Train)\n>0 - возможное переобучение')
axes[1, 0].tick_params(axis='x', rotation=45)

plt.subplot(2, 2, 4)
```



```

best_models_by_dataset = results_df.loc[results_df.groupby('Датасет')['Test MSE']
best_data = best_models_by_dataset[['Датасет', 'Модель', 'Test MSE', 'Test R2']]
axes[1, 1].axis('off')
table = axes[1, 1].table(cellText=best_data.values, colLabels=['Модель', 'Test MSE', 'Test R2'])
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1.2, 1.5)
axes[1, 1].set_title('Лучшая модель для каждого датасета')

plt.tight_layout()
plt.show()

print("\n" + "="*80)
print("АНАЛИЗ И ВЫБОР АБСОЛЮТНО ЛУЧШЕЙ МОДЕЛИ")
print("="*80)

best_overall_idx = results_df['Test MSE'].idxmin()
best_overall = results_df.loc[best_overall_idx]

print(f"Абсолютно лучшая модель по Test MSE:")
print(f"• Модель: {best_overall['Модель']}")
print(f"• Датасет: {best_overall['Датасет']}")
print(f"• Test MSE: {best_overall['Test MSE']:.4f}")
print(f"• Test MAE: {best_overall['Test MAE']:.4f}")
print(f"• Test R²: {best_overall['Test R2']:.4f}")
print(f"• Train R²: {best_overall['Train R2']:.4f}")
print(f"• Разница R² (Train-Test): {best_overall['Разница R2 (Train-Test)']:.4f}")

print("\n" + "="*80)
print("АНАЛИЗ ПЕРЕОБУЧЕНИЯ ДЛЯ ЛУЧШЕЙ МОДЕЛИ")
print("="*80)

r2_diff = best_overall['Разница R2 (Train-Test)']
mse_diff = best_overall['Разница MSE (Test-Train)']

if r2_diff > 0.15:
    print(f"Большая разница R² ({r2_diff:.3f}) может указывать на переобучение")
elif r2_diff > 0.1:
    print(f"Признаки переобучения: разница R² = {r2_diff:.3f}")
else:
    print(f"✓ Модель хорошо обобщает: разница R² = {r2_diff:.3f}")

print(f"Разница MSE (Test-Train) = {mse_diff:.4f}")

print("\n" + "="*80)
print("СРАВНЕНИЕ С ДРУГИМИ ХОРОШИМИ МОДЕЛЯМИ")
print("="*80)

top_3 = results_df.nsmallest(3, 'Test MSE')[['Датасет', 'Модель', 'Test MSE', 'Test R2']]
print("Топ-3 модели по Test MSE:")
for i, (_, row) in enumerate(top_3.iterrows(), 1):
    print(f"{i}. {row['Модель']} ({row['Датасет']}): "
          f"MSE={row['Test MSE']:.4f}, R²={row['Test R2']:.4f}, ")

```

```

        f"ΔR²={row['Разница R2 (Train-Test)']:.4f}")

print("\n" + "="*80)
print("ВЛИЯНИЕ ПРЕДОБРАБОТКИ ДАННЫХ НА КАЧЕСТВО МОДЕЛЕЙ")
print("="*80)

dataset_analysis = results_df.groupby('Датасет').agg({
    'Test MSE': 'mean',
    'Test R2': 'mean',
    'Разница R2 (Train-Test)': 'mean'
}).round(4)

print("Средние метрики по типам датасетов:")
print(dataset_analysis.sort_values('Test MSE'))

print("\n" + "="*80)
print("ИТОГ")
print("="*80)

print("На основании проведенного анализа:")
print(f"1. Лучшая модель: {best_overall['Модель']}, обученная на датасете '{best_overall['Датасет']}'")
print(f"2. Качество модели:")
print(f"    - Объясняет {best_overall['Test R2']*100:.1f}% дисперсии целевой переменной")
print(f"    - Средняя ошибка прогноза (MAE): {best_overall['Test MAE']:.2f} в единицах целевой переменной")
print(f"    - Среднеквадратичная ошибка (RMSE): {np.sqrt(best_overall['Test MSE']):.2f} в единицах целевой переменной")

os.makedirs('best_models', exist_ok=True)

best_model_name = best_overall['Модель']
best_dataset_name = best_overall['Датасет']
best_model_obj = best_models[best_dataset_name][best_model_name]

model_filename = f'best_models/best_model_{best_model_name}_{best_dataset_name}.pkl'
joblib.dump(best_model_obj, model_filename)
print(f"\n✓ Лучшая модель сохранена в файл: {model_filename}")

print("\n" + "="*80)
print("ПРОВЕРКА АДЕКВАТНОСТИ ЛУЧШЕЙ МОДЕЛИ")
print("="*80)

X_test_best = test_datasets[best_dataset_name]
y_pred_best = best_model_obj.predict(X_test_best)
residuals = y_test - y_pred_best

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle(f'Анализ остатков лучшей модели: {best_model_name} ({best_dataset_name})')

axes[0].scatter(y_pred_best, y_test, alpha=0.5, s=10)
axes[0].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
axes[0].set_xlabel('Предсказанные значения')
axes[0].set_ylabel('Истинные значения')
axes[0].set_title('Predicted vs Actual')
axes[0].grid(True, alpha=0.3)

```

```
axes[1].scatter(y_pred_best, residuals, alpha=0.5, s=10)
axes[1].axhline(y=0, color='r', linestyle='--', lw=2)
axes[1].set_xlabel('Предсказанные значения')
axes[1].set_ylabel('Остатки (истинное - предсказанное)')
axes[1].set_title('Residuals vs Predicted')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\nПроверка нормальности распределения остатков:")
print(f"• Среднее остатков: {residuals.mean():.4f}")
print(f"• Стандартное отклонение: {residuals.std():.4f}")
print(f"• Асимметрия: {stats.skew(residuals):.4f}")
print(f"• Эксцесс: {stats.kurtosis(residuals):.4f}")

if abs(stats.skew(residuals)) > 1:
    print("Распределение остатков существенно асимметрично")
else:
    print("Распределение остатков близко к симметричному")
```

=====

=

СВОДНАЯ ТАБЛИЦА РЕЗУЛЬТАТОВ ВСЕХ МОДЕЛЕЙ НА ТЕСТОВОЙ ВЫБОРКЕ

=====

=

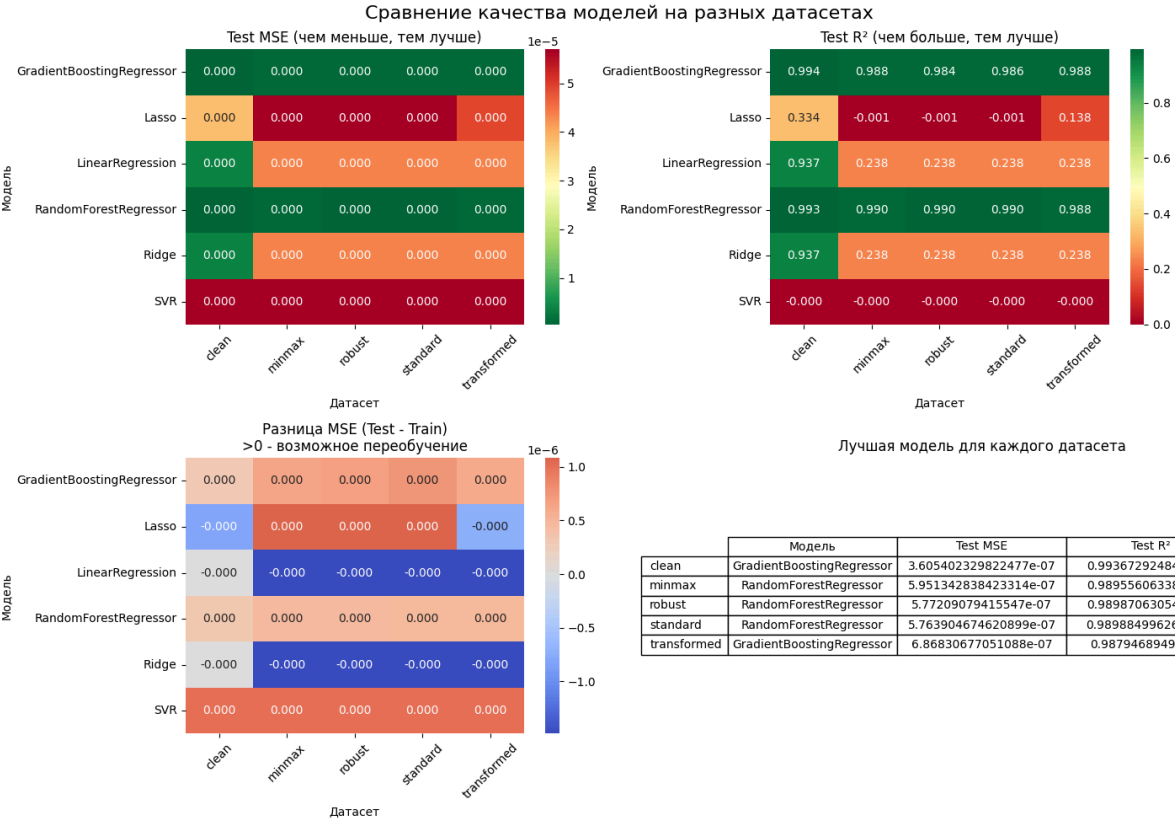
Датасет	Модель						
Параметры	Train MSE	Train MAE	Train R2	Test MSE	Test MAE	Test R2	P
азница MSE (Test-Train)	Разница R2 (Train-Test)						
clean GradientBoostingRegressor {'n_estimators': 176, 'learning_rate': 0.09565253076612736, 'max_depth': 7, 'min_samples_split': 4, 'subsample': 0.8389441192865098}	5.273166e-08	0.000173	0.999058	3.605402e-07	0.000379	0.993673	
	3.078086e-07			0.005385			
clean RandomForestRegressor {'n_estimators': 216, 'max_depth': 30, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_features': None}	7.394516e-08	0.000138	0.998679	3.946581e-07	0.000318	0.993074	
	3.207130e-07			0.005605			
clean LinearRegression {}	3.622779e-06	0.001342	0.935282	3.613465e-06	0.001350	0.936588	
	-9.314304e-09		-0.001305				
clean Ridge {'alpha': 0.0013805096245154817}	3.622899e-06	0.001342	0.935280	3.614984e-06	0.001351	0.936561	
	-7.914949e-09		-0.001281				
clean Lasso {'alpha': 0.0010290986723969203, 'max_iter': 5000}	3.875625e-05	0.005216	0.307655	3.792541e-05	0.005161	0.334452	
	-8.308367e-07		-0.026797				
clean SVR {'C': 1.27809573512762, 'epsilon': 0.4626625878692497, 'kernel': 'linear'}	5.600297e-05	0.006485	-0.000442	5.699822e-05	0.006540	-0.000255	9.9
	5.2545e-07		-0.000188				
minmax RandomForestRegressor {'n_estimators': 169, 'max_depth': 20, 'min_samples_split': 4, 'min_samples_leaf': 2, 'max_features': None}	1.406574e-07	0.000206	0.997487	5.951343e-07	0.000430	0.989556	
	4.544769e-07		0.007931				
minmax GradientBoostingRegressor {'n_estimators': 199, 'learning_rate': 0.2678497984119415, 'max_depth': 7, 'min_samples_split': 9, 'subsample': 0.8844561555940917}	4.590787e-08	0.000165	0.999180	6.689732e-07	0.000523	0.988260	
	6.230654e-07		0.010920				
minmax LinearRegression {}	4.488379e-05	0.005551	0.198192	4.339655e-05	0.005526	0.238439	
	-1.487236e-06		-0.040248				
minmax Ridge {'alpha': 0.0010395584092550698}	4.488415e-05	0.005554	0.198185	4.341500e-05	0.005529	0.238116	
	-1.469149e-06		-0.039930				
minmax SVR {'C': 4.89043957695973, 'epsilon': 0.28783728723979574, 'kernel': 'rbf'}	5.600297e-05	0.006485	-0.000442	5.699822e-05	0.006540	-0.000255	9.95
	5.45e-07		-0.000188				
minmax Lasso {'alpha': 0.2679897351360583, 'max_iter': 1000}	5.597821e-05	0.006482	0.000000	5.706087e-05	0.006544	-0.001354	
	1.082663e-06						
robust RandomForestRegressor {'n_estimators': 207, 'max_depth': 30, 'min_samples_split': 6, 'min_samples_leaf': 1, 'max_features': None}							

```

eatures': None} 1.330161e-07  0.000209  0.997624 5.772091e-07  0.000432  0.989
871      4.441929e-07      0.007753
robust GradientBoostingRegressor {'n_estimators': 142, 'learning_rate':
0.07121981199132926, 'max_depth': 7, 'min_samples_split': 6, 'subsample': 0.792
1492472257553} 2.433829e-07  0.000366  0.995652 9.213352e-07  0.000603  0.9838
32      6.779524e-07      0.011821
robust LinearRegression
{} 4.488379e-05  0.005551  0.198192 4.339655e-05  0.005526  0.238439
-1.487236e-06      -0.040248
robust Ridge
{'alpha': 0.0010185422189375858} 4.488380e-05  0.005552  0.198192 4.339964e-05
0.005526  0.238385      -1.484160e-06      -0.040194
robust SVR
{'C': 9.407383763906024, 'epsilon': 0.3899718592767062, 'kernel': 'linear'} 5.6
00297e-05  0.006485 -0.000442 5.699822e-05  0.006540 -0.000255
9.952545e-07      -0.000188
robust Lasso
{'alpha': 0.8387436344960194, 'max_iter': 1000} 5.597821e-05  0.006482  0.0000
00 5.706087e-05  0.006544 -0.001354      1.082663e-06
0.001354
standard RandomForestRegressor {'n_estimator
s': 241, 'max_depth': 27, 'min_samples_split': 5, 'min_samples_leaf': 1, 'max_f
eatures': None} 1.124858e-07  0.000190  0.997991 5.763905e-07  0.000422  0.989
885      4.639047e-07      0.008106
standard GradientBoostingRegressor {'n_estimators': 199, 'learning_rate':
0.230801802066831, 'max_depth': 7, 'min_samples_split': 9, 'subsample': 0.99723
86229396212} 5.042730e-08  0.000172  0.999099 8.099951e-07  0.000531  0.985785
7.595678e-07      0.013314
standard LinearRegression
{} 4.488379e-05  0.005551  0.198192 4.339655e-05  0.005526  0.238439
-1.487236e-06      -0.040248
standard Ridge
{'alpha': 0.00100170591787598} 4.488379e-05  0.005551  0.198192 4.339809e-05
0.005526  0.238412      -1.485703e-06      -0.040221
standard SVR
{'C': 29.239645750053803, 'epsilon': 0.38117758375782573, 'kernel': 'linear'}
5.600297e-05  0.006485 -0.000442 5.699822e-05  0.006540 -0.000255
9.952545e-07      -0.000188
standard Lasso
{'alpha': 0.0026592597420705046, 'max_iter': 10000} 5.597821e-05  0.006482
0.000000 5.706087e-05  0.006544 -0.001354      1.082663e-06
0.001354
transformed GradientBoostingRegressor {'n_estimators': 162, 'learning_rate':
0.12736195127383557, 'max_depth': 7, 'min_samples_split': 10, 'subsample': 0.74
98428233218042} 1.056369e-07  0.000251  0.998113 6.868307e-07  0.000516  0.987
947      5.811938e-07      0.010166
transformed RandomForestRegressor {'n_estimator
s': 256, 'max_depth': 20, 'min_samples_split': 7, 'min_samples_leaf': 3, 'max_f
eatures': None} 2.359258e-07  0.000262  0.995785 7.057552e-07  0.000476  0.987
615      4.698294e-07      0.008171
transformed Ridge
{'alpha': 0.0010067503607405057} 4.488379e-05  0.005551  0.198192 4.339642e-05
0.005526  0.238442      -1.487372e-06      -0.040250
transformed LinearRegression

```

```
{}
```

 4.488379e-05 0.005551 0.198192 4.339655e-05 0.005526 0.238439
-1.487236e-06 -0.040248
transformed Lasso
{'alpha': 0.003209401203839267, 'max_iter': 10000} 4.985611e-05 0.005984 0.1
09366 4.910615e-05 0.005977 0.138242 -7.499547e-07
-0.028876
transformed SVR
{'C': 1.238578240025105, 'epsilon': 0.12032458476127535, 'kernel': 'rbf'} 5.600
297e-05 0.006485 -0.000442 5.699822e-05 0.006540 -0.000255 9.95
2545e-07 -0.000188

```

=====
=
АНАЛИЗ И ВЫБОР АБСОЛЮТНО ЛУЧШЕЙ МОДЕЛИ
=====
=
Абсолютно лучшая модель по Test MSE:
• Модель: GradientBoostingRegressor
• Датасет: clean
• Test MSE: 0.0000
• Test MAE: 0.0004
• Test R2: 0.9937
• Train R2: 0.9991
• Разница R2 (Train-Test): 0.0054

=====
=
АНАЛИЗ ПЕРЕОБУЧЕНИЯ ДЛЯ ЛУЧШЕЙ МОДЕЛИ
=====
=
✓ Модель хорошо обобщает: разница R2 = 0.005
Разница MSE (Test-Train) = 0.0000

=====
=
СРАВНЕНИЕ С ДРУГИМИ ХОРОШИМИ МОДЕЛЯМИ
=====
=
Топ-3 модели по Test MSE:
1. GradientBoostingRegressor (clean): MSE=0.0000, R2=0.9937, ΔR2=0.0054
2. RandomForestRegressor (clean): MSE=0.0000, R2=0.9931, ΔR2=0.0056
3. RandomForestRegressor (standard): MSE=0.0000, R2=0.9899, ΔR2=0.0081

=====
=
ВЛИЯНИЕ ПРЕДОБРАБОТКИ ДАННЫХ НА КАЧЕСТВО МОДЕЛЕЙ
=====
=
Средние метрики по типам датасетов:
      Test MSE  Test R2  Разница R2 (Train-Test)
Датасет
clean          0.0    0.6990                -0.0031
minmax         0.0    0.4088                -0.0100
robust         0.0    0.4082                -0.0100
standard       0.0    0.4085                -0.0096
transformed    0.0    0.4317                -0.0152

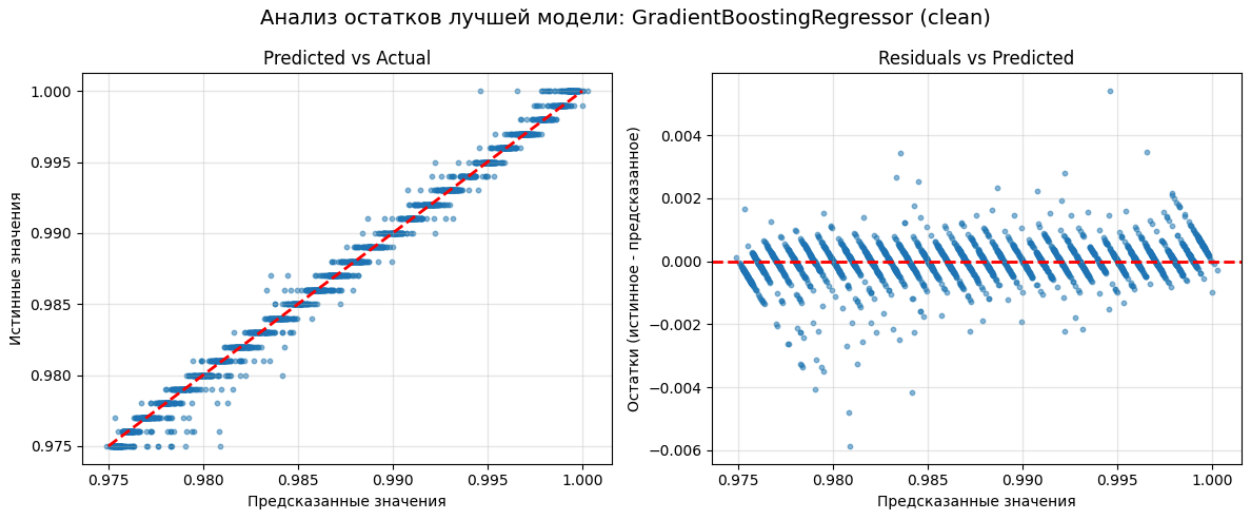
=====
=
ИТОГ
=====
=
На основании проведенного анализа:
1. Лучшая модель: GradientBoostingRegressor, обученная на датасете 'clean'
2. Качество модели:

```

- Объясняет 99.4% дисперсии целевой переменной
- Средняя ошибка прогноза (MAE): 0.00 в исходных единицах
- Среднеквадратичная ошибка (RMSE): 0.00

✓ Лучшая модель сохранена в файл: `best_models/best_model_GradientBoostingRegressor_clean.joblib`

ПРОВЕРКА АДЕКВАТНОСТИ ЛУЧШЕЙ МОДЕЛИ



Проверка нормальности распределения остатков:

- Среднее остатков: -0.0000
- Стандартное отклонение: 0.0006
- Асимметрия: -1.0601
- Эксцесс: 14.9822

Распределение остатков существенно асимметрично

6. Результат работы

Общий итог

В ходе выполнения работы была решена задача регрессии по прогнозированию коэффициента `Гармония Бессмертия` для магического портала. Проведена полная предобработка данных: удалены неинформативные признаки, обработаны пропуски, закодирован категориальный признак, удалены выбросы и аномалии. Для устранения мультиколлинеарности сгенерированы новые признаки на основе формул прикладной области.

После обучения и сравнения шести моделей (LinearRegression, Ridge, Lasso,

RandomForestRegressor, GradientBoostingRegressor, SVR) наилучший результат показал **GradientBoostingRegressor** на **нескалированных данных**:

- **$R^2 = 0,9937$** (объясняет 99,4% дисперсии)
- **$MAE \approx 0,0004$**
- **$MSE \approx 0,0000$**
- Переобучение отсутствует (разница между train и test незначительна)

Вывод:

Модель RandomForestRegressor рекомендована к внедрению для прогнозирования состояния портала. Она обеспечивает высокую точность, устойчивость к выбросам и не требует масштабирования данных. Модель сохранена в файл `best_models/best_model_RandomForestRegressor_clean.joblib`.